



HyperLink AWS Nitro TEE Rapid Review

Security Assessment

March 4, 2026

Prepared for:

Suki Sohi

HyperLink

Prepared by: **Paul Bottinelli**

Table of Contents

Table of Contents	1
Project Summary	3
Executive Summary	4
Project Goals	8
Project Targets	9
Project Coverage	10
Automated Testing	11
Summary of Findings	13
Detailed Findings	15
1. Prover enclave public key not validated against attested public key	15
2. Unauthenticated ephemeral key exchange enables parent MITM attacks	18
3. Attestation verification missing from important operator flows	21
4. Deficiencies in the enclave registration and attestation processes	23
5. KMS key policy only validates PCR0	27
6. X25519 shared secret used directly without key derivation function	29
7. Missing length validation in key exchange response may cause denial of service	31
8. Incomplete attestation document validation against AWS Nitro specification	33
9. Missing extension constraint validation in certificate chain verification	37
10. Storage download accepts unbounded allocation size	41
11. Storage rollback attack on database snapshots	43
12. Backup timestamp not validated during recovery	45
13. Potential integer truncation in database sync	47
14. Missing secret version validation may allow stale credential usage	48
15. Clock source verification only logs warning	50
16. Documented client-to-enclave encryption not implemented	52
17. Missing signature algorithm validation in X.509 chain verification	54
18. Certificate validity boundary check non-compliance with RFC 5280	56
19. Template injection vulnerabilities in GitHub Actions workflows	58
20. Unpinned external GitHub CI/CD action versions	59
21. Potential privilege escalation due to insecure Dockerfile configurations	61
22. GitHub Actions role can modify KMS key policies, bypassing attestation requirements	62
23. HTTP rate limiter configured but never invoked in request handlers	65

24. Prover proxy lacks CID validation, message size limits, and connection limits	67
A. Vulnerability Categories	69
B. Code Quality Issues	71
C. Automated Analysis Tool Configuration	74
D. Fix Review Results	76
Detailed Fix Review Results	79
E. Fix Review Status Categories	83
About Trail of Bits	84
Notices and Remarks	85

Project Summary

Contact Information

The following project manager was associated with this project:

Kimberly Espinoza, Project Manager
kimberly.espinoza@trailofbits.com

The following engineering director was associated with this project:

Jim Miller, Engineering Director, Cryptography
james.miller@trailofbits.com

The following consultant was associated with this project:

Paul Bottinelli, Consultant
paul.bottinelli@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
December 17, 2025	Pre-project kickoff call
January 12, 2026	Project check-in
January 20, 2026	Status update meeting #1
January 26, 2026	Delivery of report draft and readout meeting
January 30, 2026	Completion of fix review
March 4, 2026	Delivery of final comprehensive report with fix review

Executive Summary

Engagement Overview

HyperLink engaged Trail of Bits to review the security of its enclave application and Trusted Execution Environment (TEE) infrastructure and configuration. HyperLink is a privacy-preserving prime broker system for trading on Hyperliquid that runs within AWS Nitro Enclaves to provide secure, attestable execution with hidden balances and positions. The system combines a Rust-based enclave application with Solidity smart contracts on HyperEVM to enable private trading while maintaining cryptographic verifiability through Merkle proofs and attestation mechanisms.

One consultant conducted the review from January 12 to January 26, 2026, for a total of two engineer-weeks of effort. Our testing efforts focused on the security of the enclaves themselves and the cryptographic operations performed by the system components. With full access to source code and documentation, we performed static and dynamic testing of the codebase, using automated and manual processes. From January 29 to January 30, 2025, one consultant performed a review of the fixes provided by HyperLink, as described in [appendix D](#).

Observations and Impact

The HyperLink TEE project involves an ambitious architecture, with an extensive codebase and accompanying documentation. Our security review uncovered multiple security vulnerabilities and pervasive code quality issues, particularly within the core cryptographic and attestation mechanisms. The number and nature of the findings in this report indicate that the current implementation requires a more rigorous focus on security engineering best practices to reach the necessary level of maturity for a TEE deployment.

The impact of the findings in this report is significant, as several could lead to a compromise of the fundamental security and privacy guarantees of the system. We identified flaws in the key exchange and attestation validation processes ([TOB-HLTEE-1](#) and [TOB-HLTEE-2](#)) that could be exploited by a compromised parent VM to perform a man-in-the-middle (MitM) attack. This would allow an adversary in a privileged position to decrypt inter-enclave traffic and gain unauthorized access to sensitive user balances and positions in plaintext. No findings allowing an unprivileged attacker to compromise user's funds were identified during this review.

Another category of issues involves deficiencies in data integrity and availability mechanisms for the storage and recovery logic, including a rollback attack on database snapshots ([TOB-HLTEE-11](#)) and a denial of service by submitting an incorrect length field for the database snapshot ([TOB-HLTEE-10](#)).

The core security properties of the TEE are also undermined by several weaknesses in the deployment and attestation lifecycle. Deficiencies in the enclave registration and

attestation processes (TOB-HLTEE-4) include a reliance on PCR values being submitted by an operator on-chain, which, combined with the current lack of reproducible build and binary transparency support (also highlighted in TOB-HLTEE-4), means users cannot independently verify the legitimacy of the code running inside the TEE. Further, a high-privileged GitHub Actions role has the capability to modify the KMS key policies (TOB-HLTEE-22), thereby introducing a path for an attacker to bypass the system's intended attestation requirements entirely.

In general, the codebase contains many missing validation and security checks (TOB-HLTEE-8, TOB-HLTEE-9), as well as unused or dead code (TOB-HLTEE-12, TOB-HLTEE-14, TOB-HLTEE-23) and a documented security feature, client-to-enclave encryption, is not yet implemented (TOB-HLTEE-16). This observation further emphasizes that the codebase is still in an early stage of development.

Finally, we decreased the severities of TOB-HLTEE-1, TOB-HLTEE-2 and TOB-HLTEE-3 from high (their initial rating) to medium, as HyperLink stated that a compromise of the communication between prover enclave and client "isn't in the critical path for user fund security: all ledger logic and safeholding occurs in the main enclave."

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that HyperLink take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Implement reproducible builds and binary transparency.** Establish deterministic build processes where independent parties can build the same source commit and verify identical EIF checksums and PCR measurements. Publish build provenance metadata, including compiler versions, dependency lockfiles, and build environment details, to enable third-party verification. Maintain a transparency log of all deployed enclave images with their corresponding git commits, PCR values, and build timestamps to provide auditable evidence of what code is running in production and registered on-chain. This may also require open-sourcing the tee repository.
- **Expand test coverage with advanced testing strategies including fuzzing and negative test cases.** Implement property-based testing and fuzzing for input validation routines, CBOR parsing, and cryptographic operations to discover edge cases and input handling vulnerabilities. Add comprehensive negative test cases that verify that security controls correctly reject invalid inputs, unauthorized operations, and boundary violations. Establish minimum coverage thresholds for security-critical code paths including attestation verification, vsock message handling, and cryptographic signing logic.

- **Perform a follow-up security review with expanded scope and timeline.** This engagement covered over 50,000 lines of code in 10 person-days, limiting the depth of analysis possible for each component. Schedule a follow-up review with additional time allocated for thorough code analysis, a deployment infrastructure security assessment, and an operational security controls evaluation. Include a comprehensive review of AWS infrastructure configuration, Terraform modules, IAM policies, and production deployment procedures to validate that infrastructure-as-code matches security requirements and operational best practices.
- **Improve code quality by removing unused code and establishing maintainability standards.** The codebase is generally well-structured and documented for its complexity, but contains numerous unused fields, dead code paths, and phantom features that increase maintenance burden and create confusion about actual behavior. Conduct a systematic audit to remove unused struct fields, unreferenced functions, and unimplemented feature flags. Establish code review guidelines that prevent accumulation of dead code, require justification comments for seemingly unused code that serves a purpose, and enforce alignment between documentation and implementation to prevent future drift.
- **Integrate static analysis tools into CI/CD and establish warning remediation policies.** The tools listed in the [Automated Testing](#) section identified numerous warnings across the codebase. Configure CI/CD to run these tools on every pull request, establish policies for addressing different warning severities, and track technical debt metrics to prevent warning accumulation over time.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>
High	3
Medium	7
Low	9
Informational	5
Undetermined	0

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Access Controls	3
Auditing and Logging	1
Configuration	2
Cryptography	6
Data Exposure	1
Data Validation	8
Denial of Service	3

Project Goals

The engagement was scoped to provide a rapid security assessment of the HyperLink TEE project. Specifically, we sought to answer the following non-exhaustive list of questions:

- Can the host/enclave boundary (vsock RPC) be abused to induce unauthorized state changes?
- Can the security of AWS Nitro attestations, key registration, and KMS-attested secret loading be bypassed?
- Is the enclave signing process (EIP-712 domains, chain IDs, nonces) for orders and withdrawals correct?
- Can an adversary trigger double spend issues via the deposit/withdrawal flows, proof checks, or by replaying orders?
- Can the system state be tampered with via event sourcing, persistence, or replay attacks?
- Are the builds reproducible and the PCR0/1/2 values correctly verified during the deployment process and with the KMS policies?
- Are there any enclave risks that could lead to loss of funds or key disclosure?

Project Targets

The engagement involved reviewing and testing the following target.

TEE

Repository	https://github.com/hyperlink-xyz/tee
Version	2ccc77ad17438e22dc9aaae98f8922a6a090bd16
Type	Rust

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- Review of the documentation under `tee/docs/` as well as the [Devin AI DeepWiki](#) documentation
- Manual code review of the enclave code under `tee/enclave/src/`
- Manual code review of the host-enclave communication and input validation
- Manual code review of the AWS Nitro attestation and KMS-attested secret loading
- Manual review of the build pipeline, EIF/PCR reproducibility, and supply chain integrity
- Static code analysis using automated tools as described in [Automated Testing](#)

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. This engagement was allocated 10 person-days to review a codebase totaling over 50,000 lines of Rust code across multiple components, some complex build pipelines and GitHub Actions workflows, as well as AWS infrastructure deployment scripts. Given this time constraint, Trail of Bits performed a best-effort security review focusing on the highest-risk components and attack surfaces, as highlighted by the HyperLink team. This time limitation did not allow for a comprehensive, line-by-line review of the codebase in scope, and some areas received less scrutiny than they would in a comprehensive audit. Some system elements that may warrant further review include, but are not limited to:

- The deployment infrastructure and operational security controls, including a comprehensive review of the AWS infrastructure configuration, Terraform modules, and IAM policies
- Some of the parent-services not directly involved in the security-relevant operations listed above
- The zero-knowledge proof construction performed by the prover enclave
- The Hyperliquid exchange protocol internals, communication protocols with external blockchain services (CipherOwl screening, Fordefi custody), and the on-chain contract interaction patterns beyond the core deposit/withdrawal flows

Automated Testing

Trail of Bits uses automated techniques to extensively test the security properties of software. We use both open-source static analysis and fuzzing utilities, along with tools developed in-house, to perform automated testing of source code and compiled software.

Test Harness Configuration

We used the following tools in the automated testing phase of this project:

Tool	Description	Policy
Semgrep	An open-source static analysis tool for finding bugs and enforcing code standards when editing or committing code and during build time	Appendix C
Clippy	An open-source Rust linter used to catch common mistakes and unidiomatic Rust code	Appendix C
Dylint	A tool for running Rust lints from dynamic libraries	Appendix C
cargo-edit	A tool for quickly identifying outdated crates	Appendix C
cargo-audit	An open-source tool for checking dependencies against the RustSec advisory database	Appendix C
zizmor	A static analysis tool for GitHub Actions	Appendix C

Areas of Focus

Our automated testing and verification work focused on the following system properties:

- General code quality issues and unidiomatic code patterns
- Security of CI and GitHub actions
- Identifying dependencies that are outdated or vulnerable to known attacks

Test Results

The results of this focused testing are detailed below.

Semgrep

Running Semgrep on the codebase identified **TOB-HLTEE-21**. We did not identify any security issues using the Rust Semgrep rules. Semgrep did highlight some minor issues in portions of the codebase that were not in scope, as well as in infrastructure and deployment-related scripts. The time allotted to the project did not allow for a deeper review. Consider running Semgrep and addressing any potential security issues.

Clippy

Running Clippy on the codebase yields over 1k warnings. We recommend running Clippy with its pedantic ruleset before every major release and triaging its results. If code patterns that are identified keep recurring in the codebase, add these rules to the regular Clippy CI runs.

Dylint

Similar to Clippy above, running Dylint on the codebase identified a number of warnings that were not investigated, given the short project timeline. We recommend adding the general and supplementary rules of Dylint to the CI pipeline of the codebase.

cargo-edit

Cargo-edit identified several outdated dependencies, including some that are, in a semantic versioning sense, compatible with the version currently in use, and others that are not. Consider **integrating Dependabot** to automatically update dependencies as soon as these are available.

cargo-audit

cargo-audit did not identify any unknown security issues in the codebase's dependencies.

zizmor

Running zizmor on the codebase identified **TOB-HLTEE-19** and **TOB-HLTEE-20**. We highly recommend including zizmor in your CI/CD pipeline given the complexity of the tee GitHub actions.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Prover enclave public key not validated against attested public key	Cryptography	Medium
2	Unauthenticated ephemeral key exchange enables parent MITM attacks	Cryptography	Medium
3	Attestation verification missing from important operator flows	Cryptography	Medium
4	Deficiencies in the enclave registration and attestation processes	Cryptography	High
5	KMS key policy only validates PCR0	Access Controls	Low
6	X25519 shared secret used directly without key derivation function	Cryptography	Low
7	Missing length validation in key exchange response may cause denial of service	Denial of Service	Low
8	Incomplete attestation document validation against AWS Nitro specification	Data Validation	Medium
9	Missing extension constraint validation in certificate chain verification	Data Validation	Informational
10	Storage download accepts unbounded allocation size	Denial of Service	Medium
11	Storage rollback attack on database snapshots	Data Validation	High

12	Backup timestamp not validated during recovery	Data Validation	Informational
13	Potential integer truncation in database sync	Denial of Service	Informational
14	Missing secret version validation may allow stale credential usage	Data Validation	Low
15	Clock source verification only logs warning	Data Validation	Low
16	Documented client-to-enclave encryption not implemented	Data Exposure	Medium
17	Missing signature algorithm validation in X.509 chain verification	Data Validation	Informational
18	Certificate validity boundary check non-compliance with RFC 5280	Cryptography	Informational
19	Template injection vulnerabilities in GitHub Actions workflows	Data Validation	Low
20	Unpinned external GitHub CI/CD action versions	Auditing and Logging	Low
21	Potential privilege escalation due to insecure Dockerfile configurations	Configuration	Low
22	GitHub Actions role can modify KMS key policies, bypassing attestation requirements	Access Controls	High
23	HTTP rate limiter configured but never invoked in request handlers	Configuration	Low
24	Prover proxy lacks CID validation, message size limits, and connection limits	Access Controls	Medium

Detailed Findings

1. Prover enclave public key not validated against attested public key

Severity: Medium

Difficulty: Medium

Type: Cryptography

Finding ID: TOB-HLTEE-1

Target: enclave/src/clients/prover.rs

Description

The main enclave verifies the prover enclave's attestation and extracts the attested X25519 public key, but does not validate that subsequent key exchange operations use this attested key. The `verify_attestation` function successfully validates the prover's PCR measurements and returns the attested public key embedded in the attestation document. However, every request to the prover performs a fresh key exchange via `do_key_exchange`, and the received public key is never compared against the previously attested key.

The `verify_attestation` function validates the attestation and returns the attested public key:

```
pub async fn verify_attestation(&self) -> Result<[u8; 32]> {
    let attestation = self.get_attestation().await?;
    // ... PCR validation logic ...

    let AttestationResult {
        pcr0,
        pcr1,
        pcr2,
        timestamp_ms,
    } = verify_attestation_document(
        &attestation.attestation_doc,
        &attestation.public_key,
        false, // allow_mock
    )
    .context("Attestation verification failed")?;

    // ... PCR allowlist checks ...

    info!(
        "Prover attestation verified: PCR0={}",
        hex::encode(&pcr0[..8])
    );
}
```

```
Ok(attestation.public_key)
}
```

Figure 1.1: Function `verify_attestation` returns attested key but caller does not store it ([tee/enclave/src/clients/prover.rs](#))

When operations communicate with the prover, they call `send_request_inner`, which performs key exchange without validation:

```
async fn send_request_inner(&self, request: ProverRequest) -> Result<ProverResponse>
{
    let addr = VsockAddr::new(self.cid, self.port);
    let mut stream = VsockStream::connect(addr)
        .await
        .context("Failed to connect to proxy")?;

    // Generate keypair and do key exchange
    let (our_secret, our_public) = crypto::generate_keypair();
    let shared_secret = self
        .do_key_exchange(&mut stream, &our_secret, &our_public)
        .await?;
```

Figure 1.2: Key exchange without validation against attested key ([tee/enclave/src/clients/prover.rs](#))

The `do_key_exchange` function (called from `send_request_inner`) receives the prover's public key but never validates it, as shown in the excerpt below taken from that function:

```
let response: ProverResponse = ciborium::from_reader(&response_buf[..])?;
match response {
    ProverResponse::KeyExchange(key_resp) => {
        let their_public = PublicKey::from(key_resp.server_public_key);
        let shared = crypto::derive_shared_secret(our_secret, &their_public);
        info!("Key exchange completed with prover");
        Ok(shared)
    }
    ProverResponse::Error(e) => Err( anyhow!("Key exchange failed: {}", e)),
    _ => Err( anyhow!("Unexpected response to key exchange")),
```

Figure 1.3: Key exchange accepts any public key ([tee/enclave/src/clients/prover.rs](#))

The prover enclave generates a static X25519 keypair once at startup and embeds the public key in all attestation documents. The main enclave validates this binding during attestation verification by checking that the attestation document's `user_data` field matches the expected public key. However, the attested public key is returned to the caller and *immediately discarded*. Each subsequent key exchange operation accepts any public key provided over vsock without comparing it to the attested value.

This gap allows a compromised parent VM to intercept vsock connections and substitute its own public key during key exchange. The main enclave would establish an encrypted

session with the parent's key rather than the prover's attested key, enabling the parent to decrypt all traffic.

Exploit Scenario

An operator calls `verify_attestation` to validate the prover enclave before generating a zkPoR proof. The main enclave successfully verifies the prover's PCR measurements and extracts the attested X25519 public key from the attestation document. The main enclave logs success and discards the attested key. Ten minutes later, the `CommitStateRoot` operation sends user balance data to the prover. A compromised parent VM intercepts the vsock connection to CID 17 and proxies the connection. During key exchange, the parent responds with its own X25519 public key instead of the prover's attested key. The main enclave accepts this key without validation, derives a shared secret with the parent's key, and encrypts the user balance data. The parent decrypts the traffic, reads all user balances and positions in plaintext. The main enclave cannot detect this MitM attack because it never validated that the key exchange key matches the attested key.

Recommendations

Short term, modify the `ProverClient` structure to cache the attested public key after successful attestation verification. Add an `Option<[u8; 32]>` field to store the attested key, populate it when `verify_attestation` succeeds, and check this field in `do_key_exchange` before accepting the key exchange response. Reject connections where the received public key does not match the cached attested key, ensuring that all communications after attestation use only the verified key.

Long term, implement a robust authenticated key exchange protocol that cryptographically binds the session key to the enclave's attestation. Use a protocol such as Noise and incorporate channel binding in order for both parties to ensure they are communicating with the correct party and to make it impossible for the parent to impersonate either enclave without forging attestation signatures.

2. Unauthenticated ephemeral key exchange enables parent MITM attacks

Severity: **Medium**

Difficulty: **Low**

Type: Cryptography

Finding ID: TOB-HLTEE-2

Target: `enclave/src/clients/prover.rs`

Description

The main enclave performs a fresh unauthenticated X25519 key exchange for every request to the prover enclave, despite the prover using a static long-term key. Each request generates a new ephemeral keypair on the main enclave side and exchanges keys with the prover without any authentication mechanism. Because the parent VM acts as a transparent proxy for all vsock communication between enclaves, and because the key exchange itself is unauthenticated (see also [TOB-HLTEE-1](#)), the parent can trivially perform MitM attacks by intercepting the key exchange and substituting its own keys.

For every request to the prover, the main enclave generates a fresh key pair and performs a key exchange with the prover's public key before encrypting the request to the prover with the freshly generated shared secret, as shown below in the function `send_request_inner`:

```
async fn send_request_inner(&self, request: ProverRequest) -> Result<ProverResponse>
{
    let addr = VsockAddr::new(self.cid, self.port);
    let mut stream = VsockStream::connect(addr)
        .await
        .context("Failed to connect to proxy")?;

    // Generate keypair and do key exchange
    let (our_secret, our_public) = crypto::generate_keypair();
    let shared_secret = self
        .do_key_exchange(&mut stream, &our_secret, &our_public)
        .await?;

    // ...

    let encrypted = crypto::encrypt(&shared_secret, &request_buf)?;
```

*Figure 2.1: Key exchange without validation against attested key
(`tee/enclave/src/clients/prover.rs`)*

The key exchange protocol is completely unauthenticated. The main enclave sends its public key, receives a public key in response, and derives a shared secret without any verification. The prover enclave uses a static X25519 keypair that remains constant for its

entire lifetime. This static key is embedded in the prover's attestation document and can be verified through the attestation process. However, the key exchange protocol does not leverage this attestation binding. Instead, it accepts any public key provided over the vsock connection without verifying it matches the attested key, as also discussed in [TOB-HLTEE-1](#).

The architectural issue is compounded by the parent VM's role as a mandatory proxy for inter-enclave communication. AWS Nitro enclaves do not communicate directly via vsock; all traffic must route through the parent. The parent receives the main enclave's key exchange request in plaintext, can substitute its own public key, and can decrypt all subsequent traffic. The main enclave has no way to detect this substitution because the key exchange protocol includes no authentication mechanism.

The inefficiency of this approach is further compounded by the attestation verification workflow, which requires two successive ephemeral keypair generations and shared secret computations. When the main enclave calls `verify_attestation`, it internally calls `get_attestation`, which invokes `send_request`, which triggers `send_request_inner`, performing a complete X25519 key exchange, ECDH computation, and encrypted request-response cycle solely to retrieve the attestation document. After successfully verifying the attestation, the main enclave discards this ephemeral session entirely. When the main enclave subsequently makes an actual proof request (via `get_inclusion_proof` or similar), it calls `send_request` again, which repeats the entire key generation and exchange process from scratch. This pattern results in duplicated cryptographic operations for every attestation verification workflow, doubling the computational cost and attack surface without providing any security benefit, since both key exchanges remain equally vulnerable to MitM attacks.

Exploit Scenario

A user requests their private balance proof from the main enclave. The main enclave generates a fresh X25519 keypair and initiates key exchange with the prover enclave via vsock, which routes through the parent VM. The parent intercepts the key exchange, substitutes its own public key, and establishes two separate sessions: one with the main enclave and one with the prover. When the main enclave sends the encrypted balance query (containing user address and authentication data), the parent decrypts it, reads the sensitive query in plaintext, optionally modifies the request parameters, then re-encrypts and forwards to the prover. The prover returns the encrypted Merkle proof, which the parent again decrypts, reads (learning the user's exact balance), optionally tampers with the proof data, re-encrypts with the main enclave's session key, and forwards. The main enclave receives what appears to be a valid encrypted response and returns the potentially manipulated proof to the user. This attack succeeds on every request because the unauthenticated key exchange provides no way for either enclave to detect the parent's interposition.

Recommendations

Short term, eliminate the superfluous key exchange during the attestation verification flow and instead perform key exchange once after successful attestation verification. When `verify_attestation` succeeds, immediately establish an encrypted session with the prover using the attested public key.

Long term, implement a robust authenticated key exchange protocol that cryptographically binds the session key to the enclave's attestation. Use a protocol such as Noise and incorporate channel binding in order for both parties to ensure they are communicating with the correct party and to make it impossible for the parent to impersonate either enclave without forging attestation signatures.

3. Attestation verification missing from important operator flows

Severity: **Medium**

Difficulty: **Low**

Type: Cryptography

Finding ID: TOB-HLTEE-3

Target: `enclave/src/io/operator_handler.rs`

Description

The main enclave communicates with the prover enclave through several operator RPC endpoints, but only one endpoint verifies the prover's attestation before processing requests. The `GenerateZkporProof` operation correctly calls `verify_attestation` to validate the prover enclave's PCR measurements, timestamp freshness, and public key binding. However, the other operations (i.e., `CommitStateRoot`, `GetZkporStatus`, `VerifyZkporProof`, and `GetProofData`) skip attestation verification entirely.

For example, the `CommitStateRoot` operation is the most frequently called endpoint, executing every 10 minutes to build Poseidon Merkle trees for state commitments. This operation sends sensitive user balance and position data to the prover enclave without verifying the prover's identity.

The only operation that correctly verifies attestations is `GenerateZkporProof`:

```
OperatorRequest::GenerateZkporProof { mode } => {  
  
    // ...  
  
    if let Err(e) = client.verify_attestation().await {  
        error!(  
            "[GenerateZkporProof] Attestation verification failed: {}",  
            e  
        );  
        return RpcResponse::error(RpcError::InternalError {  
            message: format!("Prover attestation failed: {}", e),  
        });  
    }  
}
```

*Figure 3.1: GenerateZkporProof correctly verifies attestation
(tee/enclave/src/io/operator_handler.rs)*

Without attestation verification, the main enclave cannot determine whether it is communicating with the legitimate prover enclave running authorized code or with a compromised component. The prover enclave's attestation proves what specific code is

running, but skipping this check means the main enclave accepts responses from any entity that can connect via vsock.

Exploit Scenario

An operator manually triggers the `GetProofData` operation to export a zkPoR proof for external verification. The main enclave connects to the prover enclave endpoint and requests the proof data without verifying the prover's attestation. A compromised parent VM intercepts the connection and responds with fabricated proof data that appears valid. The operator receives the malicious proof, which could contain incorrect solvency claims or manipulated Merkle roots. The operator publishes this proof to users, who trust it based on the assumption that it was generated inside a verified enclave.

Recommendations

Short term, add `verify_attestation` calls at the beginning of the `CommitStateRoot`, `GetZkporStatus`, `VerifyZkporProof`, and `GetProofData` operation handlers, matching the pattern used in `GenerateZkporProof`. Reject the operation with an error if attestation verification fails. This ensures that all prover communications occur only after confirming the prover is running authorized code with valid PCR measurements.

Long term, add integration tests that verify attestation is checked for all prover endpoints and that operations fail when PCR values do not match the allowlist.

4. Deficiencies in the enclave registration and attestation processes

Severity: **High**

Difficulty: **Low**

Type: Cryptography

Finding ID: TOB-HLTEE-4

Target: `contracts/src/EnclaveKeyRegistry.sol`,
`.github/workflows/build.yml`, `parent/operator/src/main.rs`

Description

The on-chain enclave key registry accepts PCR measurements from a build-time JSON file rather than from a cryptographically signed attestation document generated by a running enclave. The build process extracts PCR0, PCR1, and PCR2 values from the Enclave Image File (EIF) metadata using `nitro-cli describe-eif` and writes them to the file `pcr-values.json`. The parent operator service reads this JSON file and submits the PCR values as plain parameters to the smart contract during key registration. The contract stores these values without cryptographic verification that they correspond to an actual running enclave's attestation document. This allows an operator to register keys from non-enclave environments or compromised enclaves.

The build process extracts PCR values from EIF metadata and writes them to a JSON file:

```
- name: Generate checksums and signature
  run: |
    sha256sum hyperlink-${{ matrix.network }}.eif > checksums-${{ matrix.network }}

    # Extract PCR values from the EIF info JSON file
    EIF_INFO="${{ steps.eif.outputs['eif-info-path'] }}"
    jq -r '.Measurements.PCR0' "$EIF_INFO" > pcr0-${{ matrix.network }}.txt

    # Generate full pcr-values.json for operator service
    jq '{pcr0: .Measurements.PCR0, pcr1: .Measurements.PCR1, pcr2:
    .Measurements.PCR2, chain_env: "${{ matrix.network }}"}' "$EIF_INFO" >
    pcr-values-${{ matrix.network }}.json
```

*Figure 4.1: Build process generates `pcr-values.json` from EIF metadata
([tee/.github/workflows/build.yml](#))*

The operator service running on the parent VM reads this JSON file when calling the function `load_pcr_values`:

```
fn load_pcr_values(implicit_path: Option<&str>) -> Result<Option<PcrValues>> {
  if let Some(path) = implicit_path {
    let data = fs::read_to_string(path).context("Failed to read PCR values
```

```

file"?)?;
    let pcrs: PcrValues = serde_json::from_str(&data).context("Failed to parse
PCR values")?;
    return Ok(Some(pcrs));
}

// Search default locations

```

Figure 4.2: Operator reads PCR values from JSON file (*tee/parent/operator/src/main.rs*)

An additional dangerous pattern exists in that function; if the JSON file containing the PCR values is not found at the explicit location specified by the parameter `explicit_path`, the function tries to load them from five hardcoded paths at default locations.

The `registerEnclave` function in the `EnclaveKeyRegistry` contract accepts these file-sourced PCR values as parameters:

```

function registerEnclave(
    address enclaveSigner,
    bytes32 pcr0,
    bytes32 pcr1,
    bytes32 pcr2,
    bytes32 x25519PublicKey,
    uint64 validFrom,
    uint64 validUntil
) external onlyOperator whenNotPaused returns (uint256) {
    // ... timing checks ...

    uint256 version = $.nextVersion++;
    _registerEnclave(version, enclaveSigner, pcr0, pcr1, pcr2, x25519PublicKey,
    validFrom, validUntil, false, false);

    return version;
}

```

Figure 4.3: PCR values accepted as operator parameters without proof
(*tee/contracts/src/EnclaveKeyRegistry.sol*)

The internal registration function (i.e., `_registerEnclave`) validates that PCR values are non-zero but never checks attestation documents. Note that the contract includes a `verifyAttestation` function that checks if provided PCR values match stored values, but this function is called only from tests and never by any production code. Cryptographic validity of the attestation is never checked.

Furthermore, the current system does not define any mechanism for users and auditors to verify that the on-chain PCR values correspond to the actual running enclave that holds the registered signing key. The protocol lacks binary transparency processes to allow third parties to do the following:

1. Build the enclave image from source and compute expected PCR values
2. Query the running enclave for its current attestation document
3. Compare the attestation's PCR values against on-chain registry values
4. Verify that the registered key matches the attestation's embedded public key

Without these verification mechanisms, the PCR values in the registry represent the build-time measurements of the ELF artifact, not proof from an actual running enclave. The gap between build-time metadata and runtime attestation undermines the security model where PCR measurements should cryptographically prove that a specific enclave binary is currently running and holds the registered signing key.

Exploit Scenario

The protocol operator controls the parent EC2 instance and has `OPERATOR_ROLE` on the `EnclaveKeyRegistry` contract. The operator downloads a legitimate release artifact containing `pcr-values-mainnet.json` from the official S3 bucket or GitHub releases. The operator generates an ECDSA keypair outside any enclave on the parent VM where they have full control and visibility. The operator modifies the operator service to submit this non-enclave signer address along with the legitimate PCR values read from `pcr-values-mainnet.json`. The smart contract accepts these parameters without requiring an attestation document from a running enclave and stores the registration with legitimate-looking PCR measurements. The operator uses the non-enclave private key to sign withdrawal authorizations and state root commitments. The contract's `validateSignature` function accepts these signatures because the signer is registered and within its validity window, never checking whether an enclave with those PCR values is actually running or holds that signing key. Users withdraw funds based on state roots signed by a key that never ran in an enclave, defeating the protocol's core security guarantee. The operator observes all user balances, orders, and positions in plaintext and manipulates state roots to steal funds. Third parties cannot detect this attack because the on-chain PCR values match the legitimate build artifact, but there is no verification that a running enclave with those measurements actually holds the registered signing key.

Recommendations

Short term, modify the registration flow to require the enclave itself to initiate key registration by generating a fresh attestation document that includes the signing key's public key in the `user_data` field and getting rid of the file-based PCR values provisioning entirely. The operator should call a new contract function `registerEnclaveWithAttestation(bytes attestationDoc)` that extracts the enclave signer address, PCR0/1/2 values, and X25519 public key from the verified attestation document rather than accepting them as parameters. Implement on-chain attestation verification that validates the NSM signature on the COSE Sign1 document, checks the certificate chain, and extracts measurements. This ensures that only keys actually running in attested enclaves can be registered.

Long term, implement more complete binary transparency and verification pipelines, allowing third parties to build the enclave EIF from source code and compute expected PCR values. Consider providing a public API endpoint where the enclave exposes its current attestation document including PCR0/1/2 and public key. Consider publishing enclave build artifacts and PCR measurements to an immutable transparency log like Certificate Transparency for auditability.

5. KMS key policy only validates PCR0

Severity: Low

Difficulty: Medium

Type: Access Controls

Finding ID: TOB-HLTEE-5

Target: docs/AWS_NITRO_SETUP.md, infrastructure/terraform/

Description

The KMS key policy validates only PCR0 (enclave image) in the attestation condition, but does not validate PCR1 or PCR2. This may allow an attacker who can modify some of the enclave's parameters to decrypt secrets from AWS Secrets Manager. The defense in depth provided by PCR1 and PCR2 validation exists in the enclave's application logic but is absent from the KMS policy that controls access to encrypted secrets.

The documented KMS key policy includes only a PCR0 condition:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Enable enclave to decrypt",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::ACCOUNT_ID:role/EC2_INSTANCE_ROLE"
      },
      "Action": "kms:Decrypt",
      "Resource": "*",
      "Condition": {
        "StringEqualsIgnoreCase": {
          "kms:RecipientAttestation:PCR0": "YOUR_PCR0_VALUE_HERE"
        }
      }
    }
  ]
}
```

Figure 5.1: KMS policy validates only PCR0 ([tee/docs/AWS_NITRO_SETUP.md#L84-L102](#))

AWS KMS supports validating all three PCR values through the `kms:RecipientAttestation:PCR1` and `kms:RecipientAttestation:PCR2` condition keys. Interestingly, in the main enclave to prover enclave attestation flow, the attestation verification code demonstrates the expected pattern by validating all three PCRs on mainnet builds:

```
verify_pcr(&pcr1, PCR1_ALLOWLIST, "PCR1"?;
verify_pcr(&pcr2, PCR2_ALLOWLIST, "PCR2"?;
```

Figure 5.2: Enclave validates PCR1 and PCR2 (in addition to PCR0 not shown above) for prover attestation ([tee/enclave/src/clients/prover.rs](#))

Without PCR1 and PCR2 validation in the KMS policy, an attacker who can control some parameters could create an enclave with the correct PCR0 (by using the legitimate enclave image) but with different PCR1 and PCR2 values. This enclave would successfully decrypt secrets from KMS because the policy checks only PCR0, despite running unauthorized kernel code or configuration.

Exploit Scenario

An operator builds the legitimate enclave image and notes the PCR0 value for the KMS policy. The KMS administrator creates a key policy that validates only PCR0, following the documentation. An attacker crafts an EIF file where bytes are carefully shifted between adjacent sections (e.g., between the kernel section and application section) such that the section boundary interpretation changes, causing different PCR1/PCR2 values while ensuring that the overall concatenated bytes remain the same, preserving PCR0. The attacker launches this enclave, which connects to the KMS proxy service and requests decryption of the secrets. KMS validates the attestation document and checks that PCR0 matches the policy condition. The PCR0 check succeeds because the enclave uses the legitimate image. KMS grants decryption without checking PCR1 or PCR2. The attacker's enclave receives the decrypted credentials and secrets in plaintext and can now exfiltrate them to the attacker.

Recommendations

Short term, consider updating the KMS key policy to validate all three PCR values using the `kms:RecipientAttestation:PCR1` and `kms:RecipientAttestation:PCR2` condition keys as a defense-in-depth best practice. The policy should require exact matches for all three measurements to ensure that only enclaves with the approved image, kernel, and configuration can decrypt secrets. Update the documentation in `docs/AWS_NITRO_SETUP.md` to show the complete policy with all three PCR conditions. When deploying new enclave versions, update all three PCR values in the KMS policy atomically to prevent rollback attacks.

Long term, implement KMS policy management through infrastructure-as-code to ensure consistent PCR validation across all environments. Create automation that extracts PCR0, PCR1, and PCR2 values from the built EIF file and generates the correct KMS policy, eliminating manual errors (as also discussed in [TOB-HLTEE-4](#)). Instead of updating the existing key policy (which requires careful coordination), consider implementing a key rotation procedure where new KMS keys are created with updated PCR values and old keys are deprecated after a transition period.

6. X25519 shared secret used directly without key derivation function

Severity: Low

Difficulty: Low

Type: Cryptography

Finding ID: TOB-HLTEE-6

Target: common/src/crypto.rs

Description

The X25519 key exchange derives a shared secret that is used directly as an AES-256-GCM encryption key without passing it through a key derivation function (KDF). Cryptographic best practices recommend applying a KDF (such as HKDF) to the raw Diffie-Hellman output before using it as a symmetric key.

The `derive_shared_secret` function returns the raw X25519 output:

```
/// Derives a shared secret from our secret key and peer's public key.
pub fn derive_shared_secret(our_secret: &StaticSecret, their_public: &PublicKey) ->
[u8; 32] {
    our_secret.diffie_hellman(their_public).to_bytes()
}
```

Figure 6.1: Raw shared secret without KDF ([tee/common/src/crypto.rs](#))

This shared secret is then used directly as the AES-256-GCM key:

```
/// Encrypts data using AES-256-GCM with the shared secret.
/// Returns: nonce (12 bytes) || ciphertext
pub fn encrypt(shared_secret: &[u8; 32], plaintext: &[u8]) -> Result<Vec<u8>> {
    let cipher = Aes256Gcm::new_from_slice(shared_secret)
        .map_err(|e| anyhow!("Failed to create cipher: {}", e))?;
```

Figure 6.2: Shared secret used directly as encryption key ([tee/common/src/crypto.rs](#))

Modern cryptographic protocols such as TLS 1.3 and the Noise Protocol Framework apply HKDF (HMAC-based Key Derivation Function) to the ECDH output before using it as a symmetric key. HKDF provides several security benefits:

1. **Uniform randomness:** The raw Diffie-Hellman output may have statistical biases. HKDF ensures the derived key is uniformly random.
2. **Domain separation:** The HKDF info parameter binds the key to a specific context (e.g., "main-to-prover-v1"), preventing key reuse across different protocols.

The missing KDF represents a deviation from cryptographic best practices that reduces defense in depth and makes the protocol less robust against future cryptanalytic advances.

Exploit Scenario

This finding does not enable a practical attack with current cryptanalysis. The impact is theoretical: if future research discovers statistical weaknesses in X25519 output distribution, or if the protocol is extended to derive multiple keys from the same shared secret without proper domain separation, the lack of KDF could become exploitable. Additionally, if the same cryptographic pattern is copied to other contexts where the shared secret might be reused, the missing KDF increases the risk of key confusion attacks.

Recommendations

Short term, wrap the `derive_shared_secret` function to apply HKDF-SHA256 before returning the key. Use the `hkdf` crate to implement HKDF with an appropriate `info` parameter that binds the key to the specific protocol context (e.g., `b"hyperlink-main-to-prover-aes-v1"`).

Long term, adopt a standardized authenticated key exchange protocol such as Noise Protocol Framework that includes built-in KDF usage, mutual authentication, as also discussed in [TOB-HLTEE-2](#).

7. Missing length validation in key exchange response may cause denial of service

Severity: Low

Difficulty: Low

Type: Denial of Service

Finding ID: TOB-HLTEE-7

Target: enclave/src/clients/prover.rs

Description

The key exchange response path does not validate the length field before allocating memory, while the regular request-response path includes proper validation. When the main enclave performs key exchange with the prover, it reads a 4-byte length field and immediately allocates a buffer of that size without checking whether the length is reasonable. A malicious response with an excessively large length value causes the enclave to attempt heap allocation that will fail and abort the process.

The key exchange path lacks length validation:

```
// Read key exchange response (unencrypted)
let mut len_buf = [0u8; 4];
AsyncReadExt::read_exact(stream, &mut len_buf).await?;
let response_len = u32::from_be_bytes(len_buf) as usize;

let mut response_buf = vec![0u8; response_len];
AsyncReadExt::read_exact(stream, &mut response_buf).await?;
```

*Figure 7.1: Key exchange response without length validation
(tee/enclave/src/clients/prover.rs#L657-L663)*

In contrast, the regular response path includes proper validation against a maximum size:

```
// Read and decrypt response
let mut len_buf = [0u8; 4];
AsyncReadExt::read_exact(&mut stream, &mut len_buf).await?;
let response_len = u32::from_be_bytes(len_buf) as usize;

if response_len == 0 {
    anyhow::bail!("Empty response from prover");
}
if response_len > MAX_RESPONSE_SIZE {
    anyhow::bail!(
        "Prover response too large: {} bytes (max {})",
        response_len,
        MAX_RESPONSE_SIZE
    );
}
```

```
    );  
}
```

*Figure 7.2: Regular response path with proper validation
([tee/enclave/src/clients/prover.rs#L617-L631](#))*

The maximum response size is defined as 50MB to accommodate large zkPoR proofs, but the key exchange response has no corresponding limit. When Rust attempts to allocate a vector larger than available memory, the allocation fails and Rust calls `handle_alloc_error`, which aborts the program. The abort is unrecoverable and requires restarting the enclave.

Exploit Scenario

The main enclave initiates a `CommitStateRoot` operation to build a Poseidon Merkle tree. The operation connects to the prover enclave via `vsock`, which routes through the parent VM. During key exchange, the parent intercepts the connection and responds with a malicious length field: `[0xFF, 0xFF, 0xFF, 0xFF]`, representing 4,294,967,295 bytes (approximately 4 GB). The main enclave reads this length value and attempts to allocate a 4 GB buffer on the heap with `vec![0u8; 4294967295]`. The enclave's memory allocator cannot satisfy this request because the enclave has far less memory available. The allocation fails, Rust invokes `handle_alloc_error`, and the enclave process aborts. The `CommitStateRoot` operation fails, the state root is not committed to the on-chain contract, and the protocol cannot update its solvency proofs until an operator manually restarts the enclave.

Recommendations

Short term, add length validation in the `do_key_exchange` function matching the pattern used in `send_request_inner`. Consider defining a single function to perform length validation in order to prevent implementation drift if one is changed and not the other. Define a maximum key exchange response size (such as 1KB, which is far larger than needed for a 32-byte public key) and reject responses that exceed this limit before attempting allocation.

Long term, add integration tests that verify the enclave rejects oversized responses and does not crash when presented with malicious length values.

8. Incomplete attestation document validation against AWS Nitro specification

Severity: **Medium**

Difficulty: **Low**

Type: Data Validation

Finding ID: TOB-HLTEE-8

Target: `enclave/src/attestation/cose.rs`

Description

The attestation document parsing code fails to enforce multiple mandatory validation rules defined in [section 3.2.2.2 of the AWS Nitro Enclaves NSM API specification](#). The specification explicitly states: "Note: If any of the previously stated rules is not fulfilled, we fail the validation." However, the current implementation omits critical validation checks, allowing malformed or non-compliant attestation documents to pass parsing and proceed to signature verification. This creates a significant attack surface where attestation documents violating the AWS specification can be accepted, potentially bypassing security controls or causing unexpected behavior.

Specifically, the AWS Nitro Enclaves NSM API specification section 3.2.2.2 defines a number of mandatory validation rules. The following ones are not correctly enforced by the implementation:

1. **Module ID:** Must be a non-empty text string
2. **Digest:** Must be present and exactly equal to "SHA384"
3. **Timestamp:** Must be greater than 0
4. **PCRs:** Must contain 1 to 32 entries, with keys in the range [0, 32), and values of 32, 48, or 64 bytes
5. **CA Bundle:** Not allowed to have 0 elements, each certificate must be 1-1024 bytes
6. **Optional parameters** (`public_key`, `user_data`, `nonce`): Must be 0-512 bytes each
7. **Certificate Size Constraints:** Must be 1-1024 bytes

These values are parsed in the function `parse_attestation_doc` in the file `tee/enclave/src/attestation/cose.rs`.

Module ID Non-Empty Check

The code extracts the `module_id` field but never validates that it is non-empty; an attestation document with `module_id: ""` will parse successfully despite violating the specification.

Digest Field

The `digest` field is completely missing from the `AttestationDocument` structure and never extracted or validated during parsing. According to the AWS Nitro Enclaves NSM API CDDL specification, the `digest` field is mandatory and must contain exactly the string "SHA384" to indicate the hash algorithm used for PCR calculations.

```
pub struct AttestationDocument {
    pub module_id: String,
    pub timestamp: u64,
    pub pcrs: PcrValues,
    pub certificate: Vec<u8>,
    pub cabundle: Vec<Vec<u8>>,
    pub user_data: Option<Vec<u8>>,
    pub nonce: Option<Vec<u8>>,
    pub public_key: Option<Vec<u8>>,
}
```

Figure 8.1: `digest` field missing from structure ([tee/enclave/src/attestation/cose.rs](#))

The parsing function extracts all other required and optional fields but never touches the `digest` field.

The AWS Nitro specification under section 2.2 defines the `digest` field as mandatory (no optional "?" prefix):

```
AttestationDocument = {
    module_id: text,
    timestamp: uint .size 8,
    digest: digest,
    pcrs: { + index => pcr },
    certificate: cert,
    cabundle: [* cert],
    ? public_key: user_data,
    ? user_data: user_data,
    ? nonce: user_data,
}

// ...

digest = "SHA384"
```

Figure 8.2: Definition of `AttestationDocument` from [section 2.2 of the specification](#)

Timestamp Greater Than Zero

The code accepts timestamp values of 0, which violates the specification requirement that timestamps must be greater than 0; an attestation document with `timestamp: 0` will parse successfully, potentially allowing replay attacks or bypassing time-based security checks.

PCR Map Constraints

The specification requires PCRs to contain 1 to 32 entries with keys in the range [0, 32). The current code does not validate the map size or key range when parsing these values with the `parse_pcrs` function.

The code does not enforce the following conditions:

- Map must have 1–32 entries (could have 0 entries or 100 entries)
- Keys must be in range [0, 32) (could have key 50 or 1000)

An attestation document with an empty PCR map or with PCR keys outside the valid range will parse successfully. Additionally, the function `get_pcr_by_index`, which is used to parse specific values from the map, returns `None` in case the entry is missing.

CA Bundle Non-Empty Check During Parsing

While `verify_cert_chain` in `verification.rs` checks that the CA bundle is non-empty (line 187), the parsing function itself does not enforce this constraint. The specification requires validation to occur during parsing. Each certificate in the bundle should also be in the range [1, 1024].

Optional Parameter Size Limits

The specification limits optional parameters (`user_data`, `nonce`) to 0–512 bytes each and the `public_key` to 1–1024 bytes. The current code does not enforce these size limits. An attestation document with a 10 KB `user_data` field or a 5 MB `nonce` will parse successfully despite violating the specification's byte limit per field.

Certificate Size Constraints

The specification limits the size of the `certificate` field to 1–1024 bytes. The current code does not enforce this limit.

Attestation Size

In addition, the COSE Sign1 parsing function (i.e., `parse_cose_sign1`) does not enforce the overall AWS Nitro Enclaves attestation document size limit before parsing and allocating memory. According to the AWS Nitro Enclaves NSM API specification, attestation documents have a maximum size of 16 KiB (16,384 bytes); see the below excerpt from [section 3.1 of the specification](#).

```
18(/* COSE_Sign1 CBOR tag is 18 */
```

```

{1: -35}, /* This is equivalent with {algorithm: ECDS 384} */
}, /* We have nothing in unprotected */
bstr .size (1..16384), /* Attestation Document */
bstr .size (32/48/64) /* Signature */
)

```

Figure 8.3: Allowed attestation size from [section 3.1 of the specification](#)

Exploit Scenario

An attacker exploits the missing digest validation by crafting an attestation document where the digest field is set to "SHA512" but PCR values are 48 bytes (truncated from 64-byte SHA-512 hashes). The parser accepts this document because it never checks the digest field. The PCR validation logic operates on data with ambiguous semantics, potentially comparing truncated SHA-512 PCR values against expected SHA-384 values in allowlists, leading to incorrect security decisions about which enclaves to trust.

Recommendations

Short term, implement all mandatory validation rules from AWS Nitro specification section 3.2.2.2 in the `parse_attestation_doc` function and enforce the maximum size from section 3.1 in the `parse_cose_sign1` function. Return clear, specific error messages for each validation failure to aid debugging and security monitoring. These checks should occur before any memory-intensive operations and before signature verification, ensuring that non-compliant documents are rejected as early as possible.

Long term, consider creating a formal attestation document validator that can be unit tested against known-good and known-bad attestation documents from AWS documentation. Add property-based testing (using proptest or similar) to generate random attestation documents and verify that all specification-violating documents are rejected. Add comprehensive tests that verify the parser rejects documents with missing digest fields, wrong digest values, and mismatched digest-PCR length combinations.

References

- [AWS Nitro Enclaves NSM API - Attestation Process \(Section 3.2.2.2\)](#)
- [RFC 8152 - CBOR Object Signing and Encryption \(COSE\)](#)

9. Missing extension constraint validation in certificate chain verification

Severity: Informational

Difficulty: Low

Type: Data Validation

Finding ID: TOB-HLTEE-9

Target: enclave/src/attestation/verification.rs

Description

The certificate chain verification code does not validate critical certificate extension constraints as required by the [AWS Nitro Enclaves specification](#) sections 3.2.3.2 and 3.2.3.3. Specifically, the code fails to validate (1) the `pathLenConstraint` field in the Basic Constraints extension and (2) the absence of `keyCertSign` in the Key Usage extension for leaf certificates. These missing validations allow certificate chains that violate X.509 standards and the AWS Nitro specification to pass verification, potentially accepting malformed or malicious certificates.

`pathLenConstraint` in Basic Constraints

The AWS specification requires that root and intermediate certificates must have `pathLenConstraint` values greater than or equal to the remaining chain length, and that leaf certificates must not have `pathLenConstraint` defined. The current implementation only checks whether CA certificates have `CA=true` and whether leaf certificates have `CA=false`, but never validates the `pathLenConstraint` field.

The `verify_ca_extensions` function checks Basic Constraints but ignores `pathLenConstraint`:

```
fn verify_ca_extensions(cert: &X509Certificate, cert_name: &str) -> Result<()> {
    // Check Basic Constraints extension for CA=true
    if let Ok(Some(bc)) = cert.basic_constraints() {
        if !bc.value.ca {
            return Err(anyhow!("{} has Basic Constraints with CA=false",
cert_name));
        }
    } else {
        return Err(anyhow!("{} missing Basic Constraints extension", cert_name));
    }

    // Check Key Usage extension for keyCertSign
    if let Ok(Some(ku)) = cert.key_usage() {
        if !ku.value.key_cert_sign() {
            return Err(anyhow!(
                "{} missing keyCertSign in Key Usage extension",
                cert_name
            ));
        }
    }
}
```

```

    ));
  }
} else {
    return Err(anyhow!("{}", cert_name).missing(KeyUsageExtension));
}

Ok(())
}

```

Figure 9.1: CA extension verification missing pathLenConstraint check
([tee/enclave/src/attestation/verification.rs#L303-L326](#))

The `verify_leaf_extensions` function checks that the leaf certificate is not a CA but does not verify the absence of `pathLenConstraint`:

```

/// Verify that a leaf certificate is NOT a CA and has digitalSignature key usage.
fn verify_leaf_extensions(cert: &X509Certificate) -> Result<> {
    // Leaf certificate must not be a CA
    if let Ok(Some(bc)) = cert.basic_constraints() {
        if bc.value.ca {
            return Err(anyhow!(
                "Attestation certificate must not be a CA certificate"
            ));
        }
    }
    // Missing Basic Constraints is acceptable for leaf certs (defaults to non-CA)

    // Check Key Usage extension for digitalSignature - required for AWS attestation
    certs
    if let Ok(Some(ku)) = cert.key_usage() {
        if !ku.value.digital_signature() {
            return Err(anyhow!(
                "Attestation certificate missing digitalSignature in Key Usage"
            ));
        }
    } else {
        return Err(anyhow!(
            "Attestation certificate missing required Key Usage extension"
        ));
    }

    Ok(())
}

```

Figure 9.2: Leaf extension verification missing pathLenConstraint and keyCertSign checks
([tee/enclave/src/attestation/verification.rs#L274-L300](#))

The X.509 standard (RFC 5280 section 4.2.1.9) states that the `pathLenConstraint` field “gives the maximum number of non-self-issued intermediate certificates that may follow this certificate in a valid certification path.” If a certificate has `pathLenConstraint` set to 0, it can issue only end-entity certificates, not CA certificates. If set to 1, it can issue CA

certificates that themselves can issue only end-entity certificates. The absence of this field means there is no limit.

keyCertSign Prohibition for Leaf Certificates

The AWS specification section 3.2.3.3 explicitly states the `keyCertSign` bit “may only be asserted in CA certificates.” The current implementation only checks that the `digitalSignature` bit is present but does not verify that `keyCertSign` is absent. This allows leaf attestation certificates with inappropriate key usage permissions to pass validation.

The prohibition against `keyCertSign` in leaf certificates is a fundamental X.509 security requirement. The `keyCertSign` bit indicates that the certificate’s public key may be used to verify signatures on certificates, which is only appropriate for CA certificates. RFC 5280 section 4.2.1.3 states that: “If the `keyCertSign` bit is asserted, then the `ca` bit in the basic constraints extension MUST also be asserted.” This creates a consistency requirement between Basic Constraints and Key Usage.

While AWS Nitro certificates likely conform to the specification in practice, the missing validation means the code does not enforce the security guarantees defined by the X.509 standard and AWS specification. This means that the code accepts malformed or malicious certificates that should be rejected.

Recommendations

Short term, implement all missing constraint validations according to AWS Nitro specification sections 3.2.3.2 and 3.2.3.3.

Long term, implement comprehensive X.509 consistency validation and testing:

1. Add test cases covering all constraint scenarios:
 - 1.1. Root certificate with `pathLenConstraint` less than chain length (should be rejected)
 - 1.2. Intermediate certificate with `pathLenConstraint=0` followed by another CA (should be rejected)
 - 1.3. Leaf certificate with any `pathLenConstraint` value (should be rejected)
 - 1.4. Leaf certificate with `CA=false` and `keyCertSign=true` (should be rejected)
 - 1.5. Certificates without `pathLenConstraint` (None) accepted regardless of chain depth
2. Document the relationships between certificate extensions:
 - 2.1. `CA=true` requires `keyCertSign=true`

- 2.2. CA=false requires keyCertSign=false
- 2.3. pathLenConstraint must align with actual chain depth
- 2.4. Leaf certificates must not define pathLenConstraint

References

- [AWS Nitro Enclaves NSM API](#)
- [RFC 5280 - Section 4.2.1.9: Basic Constraints](#)
- [RFC 5280 - Section 4.2.1.3: Key Usage](#)
- [X.509 Certificate Path Validation](#)

10. Storage download accepts unbounded allocation size

Severity: **Medium**

Difficulty: **Medium**

Type: Denial of Service

Finding ID: TOB-HLTEE-10

Target: `enclave/src/persistence/sync.rs`

Description

The enclave accepts database snapshots from an external S3 bucket via the parent storage service during startup and recovery operations. The `download_redb` function reads a 32-bit length field from the parent and immediately allocates a buffer of that size without enforcing maximum bounds. A malicious parent can send a length value up to 4 GB (the maximum value of a 32-bit unsigned integer), causing the enclave to exhaust available memory and crash.

The vulnerable code reads the data length from the untrusted parent and allocates memory based on that value:

```
let mut len_buf = [0u8; 4];
stream.read_exact(&mut len_buf).await?;
let data_len = u32::from_le_bytes(len_buf) as usize;

if data_len == 0 {
    anyhow::bail!("Received empty redb data");
}

let mut data = vec![0u8; data_len];
stream.read_exact(&mut data).await?;
```

*Figure 10.1: Unbounded memory allocation in storage download
(`tee/enclave/src/persistence/sync.rs`)*

The only validation performed is checking whether the length is zero. No maximum size limit is enforced before the allocation occurs. Because AWS Nitro Enclaves have constrained memory resources (4 GB in the current deployment setup), a malicious parent can easily trigger an out-of-memory condition.

Exploit Scenario

The enclave is restarted and attempts to recover its previous state by downloading its database snapshot from the parent storage service. A compromised parent operator intercepts this request and responds with a storage sync message containing `data_len = 0xFFFFFFFF` (4,294,967,295 bytes, or 4 GB). The enclave immediately attempts to allocate 4 GB of memory. Because this would use up all of the enclave's available memory, the Rust

allocator fails and the enclave panics. The system remains unavailable until infrastructure operators manually intervene to replace the compromised parent service.

Recommendations

Short term, add a maximum size constant and reject download requests exceeding this limit before allocation. Set the limit based on expected database size (e.g., 100 MB).

Long term, consider processing the database in chunks rather than allocating the full size upfront. Add comprehensive testing such as fuzz testing to trigger such denial-of-service vectors.

11. Storage rollback attack on database snapshots

Severity: **High**

Difficulty: **Medium**

Type: Data Validation

Finding ID: TOB-HLTEE-11

Target: `enclave/src/persistence/sync.rs`

Description

The enclave loads its database file from storage controlled by a potentially untrusted parent during startup and recovery operations. The database file is stored in S3 by the parent storage service, which has complete control over which version is served to the enclave. The enclave has no cryptographic mechanism to verify that it received the latest database state or to detect when the parent provides an outdated file.

The storage sync protocol operates across the TEE boundary via vsock, with the parent acting as intermediary to external storage. The database file is downloaded and decrypted in the function `download_redb`:

```
pub async fn download_redb(
    dest_path: &Path,
    encryptor: Option<&StorageEncryptor>,
) -> Result<bool> {
    debug!("Attempting to download redb from storage service");

    // ...

    // Read timestamp, checksum, data

    // ...

    // Verify checksum of received (possibly encrypted) data
    let computed: [u8; 32] = Sha256::digest(&data).into();
    if computed != checksum {
        anyhow::bail!(
            "Checksum mismatch: expected {}, got {}",
            hex::encode(&checksum[..8]),
            hex::encode(&computed[..8])
        );
    }

    // Optionally decrypt
    let plaintext = match encryptor {
        Some(enc) => {
            let decrypted = enc.decrypt(&data).context("Failed to decrypt redb")?;
            debug!(
```

```

        "Decrypted redb: {} -> {} bytes",
        data.len(),
        decrypted.len()
    );
    decrypted
}
None => data,
};

```

*Figure 11.1: Database file loaded from untrusted parent storage
([tee/enclave/src/persistence/sync.rs](#))*

The enclave validates data integrity via SHA-256 checksum (which, in this case, does not really provide any additional security compared to the underlying database encryption) but has no mechanism to verify temporal validity. The parent storage service can serve any historical version of the database file stored in S3. The timestamp provided by the parent (line 168) is informational only and currently not validated against any trusted source (see also finding [TOB-HLTEE-12](#)). This violates the TEE security model where code inside the enclave should not trust data from outside the TCB without cryptographic proof of authenticity and freshness.

Exploit Scenario

The enclave crashes and requests database recovery from the parent storage service. A compromised parent operator retrieves an old database file from S3 taken before recent deposits and serves it to the enclave instead of the latest version. The enclave loads the old state and begins processing, now operating with balances that conflict with the on-chain committed state root. Users whose deposits occurred shortly before the crash see incorrect balances despite valid on-chain deposit proofs, and their withdrawal requests fail validation, effectively locking funds until operators manually restore correct state.

Recommendations

Short term, add the timestamp to the AAD field of the AEAD scheme and validate it when downloading and decrypting a snapshot.

Long term, also include the block number and latest state root Merkle hash as an AAD in the encryption process when saving the database state. Query the `StateManager` contract during recovery to ensure that the snapshot's state root and block numbers are actually consistent with the latest ones on-chain.

12. Backup timestamp not validated during recovery

Severity: Informational

Difficulty: Undetermined

Type: Data Validation

Finding ID: TOB-HLTEE-12

Target: enclave/src/persistence/sync.rs, enclave/src/init/engine.rs

Description

The enclave's remote persistence protocol includes a timestamp field in backup metadata that is transmitted during upload and download operations but is never validated. The timestamp is included in the `RedbBackupMetadata` structure and sent over the vsock connection to the storage service. However, the enclave only logs this timestamp and does not perform any validation checks, such as attempting to detect stale backups.

During backup upload, the enclave obtains the current system time and includes it in the protocol message sent to the storage service:

```
// Get current timestamp
let timestamp = std::time::SystemTime::now()
    .duration_since(std::time::UNIX_EPOCH)
    .context("Failed to get system time")?
    .as_secs();

debug!(
    "Uploading redb: {} bytes, checksum={}, ts={}",
    data_len,
    hex::encode(&checksum[..8]),
    timestamp
);
```

Figure 12.1: Timestamp included in backup upload
(tee/enclave/src/persistence/sync.rs)

During recovery, the enclave retrieves the timestamp from the backup metadata but uses it only for logging purposes. The `initialize_storage` function calls `check_redb_exists` to retrieve metadata and logs the timestamp value, but no validation occurs:

```
match StorageSyncClient::check_redb_exists().await {
    Ok(Some(meta)) => {
        info!("Backup found: ts={}, size={}", meta.timestamp, meta.size);
        match StorageSyncClient::download_redb(&redb_path, encryptor).await {
            Ok(true) => info!("Restored from storage service"),
            Ok(false) => anyhow::bail!("Backup check succeeded but download empty"),
            Err(e) => anyhow::bail!("Backup download failed: {}", e),
        }
    }
}
```

```

    }
  }
  Ok(None) => info!("No backup found - fresh deployment"),
  Err(e) => anyhow::bail!("Storage service unreachable: {} (fail-closed)", e),
}

```

*Figure 12.2: Timestamp logged but not validated during recovery
(tee/enclave/src/init/engine.rs)*

The timestamp field provides no defensive value. The storage service runs on the parent VM, which is outside the trust boundary, so a compromised parent could serve an old backup with a valid checksum (as discussed in finding [TOB-HLTEE-11](#)).

Recommendations

Short term, add the timestamp to the AAD field of the AEAD scheme and validate it when downloading and decrypting a snapshot.

Long term, include the block number and latest state root Merkle hash as an AAD in the encryption process when saving the database state. Query the StateManager contract during recovery to ensure that the snapshot's state root and block numbers are actually consistent with the latest ones on-chain.

13. Potential integer truncation in database sync

Severity: **Informational**

Difficulty: **Medium**

Type: Denial of Service

Finding ID: TOB-HLTEE-13

Target: `enclave/src/persistence/sync.rs`

Description

The database sync protocol silently truncates the database file size from `usize` to `u32` when uploading backups to the storage service. If the `redb` database grew beyond 4 GB, the truncated length would cause protocol desynchronization between the enclave and storage service, preventing successful backups and potentially causing data loss on enclave restart. In the current deployment, the enclave is provisioned with 4 GB of total memory; it would thus crash before reaching this limit.

The enclave reads the entire database file into memory and sends its length to the storage service using a 4-byte length prefix:

```
timeout(SYNC_TIMEOUT, async {
    stream.write_u8(STORE_REDB).await?;
    stream.write_all(&timestamp.to_le_bytes()).await?;
    stream.write_all(&checksum).await?;
    stream.write_all(&(data_len as u32).to_le_bytes()).await?;
    stream.write_all(&data).await?;
    stream.flush().await?;
    Ok::<_, anyhow::Error>(())
})
```

*Figure 13.1: Database size truncated from `usize` to `u32`
(`tee/enclave/src/persistence/sync.rs`)*

Rust's `as` cast operator performs silent truncation without overflow checks in both debug and release builds. When casting from `usize` to `u32` on 64-bit systems, values exceeding `u32::MAX` (4,294,967,295 bytes) wrap around.

Recommendations

Short term, upgrade the sync protocol to use 8-byte length prefixes (`u64`) to accommodate databases larger than 4 GB. Change both the enclave sender and storage service receiver to use `u64` length fields

Long term, add comprehensive testing such as fuzz testing to trigger such edge cases.

14. Missing secret version validation may allow stale credential usage

Severity: Low

Difficulty: Medium

Type: Data Validation

Finding ID: TOB-HLTEE-14

Target: `enclave/src/init/services.rs`,
`enclave/src/core/attestation_loader.rs`, `enclave/src/core/secrets.rs`

Description

The enclave loads secrets from AWS Secrets Manager via KMS attestation but does not validate the version field in the `EnclaveSecrets` structure. The version field is defined in the structure and logged during secret loading, but no validation logic checks whether the loaded version is current, expected, or above a minimum threshold. This allows a compromised parent VM to force the enclave to use outdated secret versions that may contain rotated or revoked credentials.

The secret loading flow operates as follows: the enclave requests secrets from the parent's KMS proxy service, which fetches encrypted data from AWS Secrets Manager and forwards it to AWS KMS for attestation-based re-encryption. The parent KMS proxy controls which secret version is retrieved from Secrets Manager. Although AWS Secrets Manager supports multiple concurrent secret versions, the enclave accepts any version without verification, as shown in the figure below.

```
/// Load secrets via KMS attestation
async fn load_secrets() -> Result<EnclaveSecrets> {
    let loader =
        AttestationLoader::with_default_port().context("Failed to create
AttestationLoader")?;

    let secrets = loader
        .load_secrets(SECRET_ID)
        .await
        .context("Failed to load secrets from KMS")?;

    if secrets.fordefi.is_none() {
        bail!("Fordefi credentials missing from secrets");
    }

    info!("Secrets loaded (version {})", secrets.version);
    Ok(secrets)
}
```

Figure 14.1: Secret loading without version validation
(`tee/enclave/src/init/services.rs`)

The `EnclaveSecrets` structure defines the version field with a default value, but the field serves only observability purposes:

```
/// All enclave secrets in a single structure.
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct EnclaveSecrets {
    /// ...

    /// Version for rotation tracking
    #[serde(default)]
    pub version: u32,
}
```

Figure 14.2: `EnclaveSecrets` structure with unused version field
([tee/enclave/src/core/secrets.rs](#))

While the ephemeral keypair generated per enclave boot prevents replay of captured KMS responses across different enclave sessions, it does not prevent a compromised parent from requesting specific old versions that still exist in AWS Secrets Manager. The parent can modify the `get_secret_value` call to include a `version_id` parameter, causing Secrets Manager to return an older secret version. Since the enclave performs no version validation, it will accept and use the outdated credentials.

Exploit Scenario

An operator rotates the Fordefi and CipherOwl credentials from version 1 to version 2 after suspecting a compromise. The operator updates the secret in AWS Secrets Manager with new credentials and increments the version field to 2. However, AWS Secrets Manager retains version 1 to support graceful rollback.

Eve, an attacker who has compromised the parent EC2 instance, modifies the KMS proxy service code to request version 1 to the Secrets Manager API call. When the enclave boots and requests secrets, the modified KMS proxy fetches version 1 instead of version 2. The enclave receives the KMS-decrypted version 1 secrets without error, and proceeds to use the old, potentially compromised credentials.

Recommendations

Short term, add version validation in the `load_secrets` function to reject secrets below a minimum required version. Define a `MINIMUM_SECRET_VERSION` constant that operators increment when rotating secrets, and compare the loaded version against this threshold.

Long term, embed the expected secret version as an environment variable during enclave build time, causing it to be included in the PCR measurements. Update the KMS key policy to validate all PCR values (also discussed in [TOB-HLTEE-5](#)), which cryptographically enforces that only enclaves built for a specific secret version can decrypt those secrets. Add integration tests that verify version validation logic rejects old versions and that the enclave fails to start when version mismatches occur.

15. Clock source verification only logs warning

Severity: Low

Difficulty: Low

Type: Data Validation

Finding ID: TOB-HLTEE-15

Target: `enclave/src/core/time_sync.rs`

Description

The `verify_clock_source` function checks whether the enclave uses `kvm-clock` as its clock source but only logs a warning when a different clock source is detected. The function does not enforce the use of `kvm-clock` or prevent the enclave from continuing operation with an unexpected clock source.

AWS Nitro Enclaves **should use `kvm-clock` for reliable timekeeping** within the virtualized environment. The function `verify_clock_source` reads the current clock source from `/sys/devices/system/clocksource/clocksource0/current_clocksource` and compares it to the expected value of `"kvm-clock"` or `"kvm-clocksource"`. When a different clock source is active, the function only logs a warning but returns as normal without raising an error:

```
pub(crate) fn verify_clock_source() {
    const CLOCK_PATH: &str =
"/sys/devices/system/clocksource/clocksource0/current_clocksource";

    let Ok(contents) = std::fs::read_to_string(CLOCK_PATH) else {
        tracing::info!("Clocksource sysfs not present; skipping verification");
        return;
    };

    let current = contents.trim();

    if current == "kvm-clock" || current == "kvm-clocksource" {
        tracing::info!(clock_source = current, "Clock source verified");
    } else {
        tracing::warn!(
            clock_source = current,
            "Clock source is not kvm-clock; drift limiting will mitigate"
        );
    }
}
```

Figure 15.1: Clock source verification only logs warning
(`tee/enclave/src/core/time_sync.rs`)

The function's name suggests that it performs verification that could fail, but the implementation treats all outcomes as non-fatal. When the `sysfs` file is unavailable, the function logs an informational message and returns. When an unexpected clock source is detected, the function logs a warning and returns. In both cases, the enclave continues initialization without any indication to the caller that verification failed.

Exploit Scenario

An operator deploys the enclave on a misconfigured EC2 instance where the kernel boots with TSC as the active clock source instead of `kvm-clock`. The enclave starts successfully and the `verify_clock_source` function logs a warning to the console. The operator does not monitor enclave startup logs closely and misses this warning. Over time, the enclave starts rejecting valid requests when the clock drifts beyond acceptable thresholds.

Recommendations

Short term, rename `verify_clock_source` to `check_clock_source` or `log_clock_source` to accurately reflect that it performs logging rather than verification with enforcement. This prevents misleading expectations about the function's behavior and clarifies that clock source checking is informational rather than mandatory.

Long term, consider whether the enclave should fail initialization when running with an unexpected clock source. If `kvm-clock` is critical for reliable operation in Nitro Enclaves, change the function to return a `Result` and propagate clock source validation failures to the caller.

16. Documented client-to-enclave encryption not implemented

Severity: **Medium**

Difficulty: **Low**

Type: Data Exposure

Finding ID: TOB-HLTEE-16

Target: `enclave/src/io/engine_handler.rs`, `enclave/src/crypto/mod.rs`, `docs/ARCHITECTURE.md`, `docs/TESTING.md`, `docs/ENCLAVE_ARCHITECTURE.md`, `docs/SMART_CONTRACT_DESIGN.md`, `docs/TEE_MONITORING.md`

Description

The system documentation claims that client-to-enclave communication uses X25519 Diffie-Hellman key exchange with AES-256-GCM encryption for confidentiality, but this encryption is not implemented. All exchange requests, including orders, withdrawals, and transfers, are transmitted as plaintext CBOR over vsock, allowing the parent VM to observe all user activity.

The documentation states in multiple locations that end-to-end encryption protects client communication. The `ARCHITECTURE.md` file claims that “E2E Encryption: Client communication uses X25519 + AES-GCM,” and the `TESTING.md` file describes the X25519 public key as enabling “E2E encrypted communication with enclave.” However, the implementation does not perform encryption at any point in the client-to-enclave communication path. When a client communicates with the enclave, the TLS connection terminates at CloudFront (outside the enclave), and all subsequent communication from the parent to the enclave is in plaintext.

The enclave generates an X25519 keypair during initialization and returns the public key in handshake responses, but the corresponding secret key is marked as dead code and never used:

```
#[derive(Clone)]
pub struct EnclaveKeys {
    pub signing_key: SigningKey,
    #[allow(dead_code)]
    x25519_secret: X25519Secret,
    pub x25519_public: X25519Public,
    pub pending_keys: Option<PendingKeySet>,
}
```

Figure 16.1: X25519 secret key marked as dead code (`tee/enclave/src/crypto/mod.rs`)

The handshake handler receives an optional client public key but ignores it completely, never deriving a shared secret:

```
Request::Handshake { client_pub_key: _ } => {
    let response = RpcResponse::ok(HandshakeResponse {
        enclave_pub: self.x25519_public,
        enclave_signer: self.signer_address,
        session_id: None,
        success: true,
    });
    ciborium::into_writer(&response, write_buf)?;
}
```

*Figure 16.2: Client public key ignored in handshake
(tee/enclave/src/io/engine_handler.rs)*

While EIP-712 signatures prevent tampering and fabrication of user actions, they provide no confidentiality. The parent VM can read all order details, including asset, size, price, direction, withdrawal amounts, destinations, and transfer recipients. This contradicts the documented security model, which explicitly claims encryption protects client communication.

Exploit Scenario

Alice believes her trading activity is protected by end-to-end encryption based on the system documentation. She places a large market order through the Hyperlink platform. The order request travels from her client to the relay as HTTPS-encrypted JSON, but once it reaches the relay, the parent VM can read the plaintext order details before forwarding to the enclave over vsock. Eve, who has compromised the parent VM, observes Alice's order in plaintext CBOR format, including the exact size, price, and direction. Eve front-runs Alice's order on Hyperliquid using this information, extracting value from Alice's trade. Alice loses money on the trade and remains unaware that her supposedly encrypted order was visible to the parent VM.

Recommendations

Short term, update all documentation to accurately reflect the current security model. Remove claims about X25519 and AES-GCM encryption for client-to-enclave communication. Document that the parent VM can observe all exchange request data.

Long term, either implement the documented encryption or remove the vestigial X25519 keypair from `EnclaveKeys` to eliminate confusion. If implementing encryption, derive a shared secret from the client and enclave public keys during the handshake, use it to encrypt exchange request payloads with AES-256-GCM, and decrypt them in the enclave before processing.

17. Missing signature algorithm validation in X.509 chain verification

Severity: Informational

Difficulty: Low

Type: Data Validation

Finding ID: TOB-HLTEE-17

Target: `enclave/src/attestation/verification.rs`

Description

The certificate chain verification code does not validate the signature algorithm field in X.509 certificates before using them for signature verification. The code extracts the public key and verifies that it uses ECDSA P-384, and the signature verification is performed using SHA-384, but it never validates that the certificate's `signatureAlgorithm` field matches the expected `ecdsa-with-SHA384` algorithm.

In practice, AWS Nitro Enclaves issue certificates with SHA-384, so this validation gap does not pose a security risk in the current deployment. However, the code does not explicitly validate this assumption, creating a deviation from X.509 best practices.

The `verify_cert_signature` function performs signature verification without checking the algorithm:

```
fn verify_cert_signature(subject: &X509Certificate, issuer: &X509Certificate) ->
Result<> {
    let issuer_key =
        extract_public_key_from_x509(issuer).context("Failed to extract issuer
public key")?;

    let sig_bytes = subject.signature_value.data.as_ref();
    let signature = parse_p384_signature(sig_bytes).context("Invalid certificate
signature")?;

    let tbs_bytes = subject.tbs_certificate.as_ref();
    let digest = Sha384::digest(tbs_bytes);

    issuer_key
        .verify_prehash(&digest, &signature)
        .map_err(|e| anyhow!("Certificate signature verification failed: {}", e))
}
```

Figure 17.1: Certificate signature verification without algorithm validation
(`tee/enclave/src/attestation/verification.rs`)

According to X.509 standards (RFC 5280), certificates contain two `signatureAlgorithm` fields that should be validated:

1. `tbsCertificate.signature`: The algorithm identifier for the signature that the CA uses to sign the certificate
2. `certificate.signatureAlgorithm`: Repeated at the certificate level (must match the field in `tbsCertificate`)

For AWS Nitro Enclaves, the expected algorithm is `ecdsa-with-SHA384` (OID 1.2.840.10045.4.3.3). While AWS consistently uses this algorithm, the code does not validate this requirement.

Recommendations

Short term, add validation of the certificate signature algorithm in the `verify_cert_signature` function to make the AWS assumption explicit.

Long term, add comprehensive tests to exercise the certificate validation.

18. Certificate validity boundary check non-compliance with RFC 5280

Severity: Informational

Difficulty: Low

Type: Cryptography

Finding ID: TOB-HLTEE-18

Target: `enclave/src/attestation/verification.rs`

Description

The `verify_cert_validity` function incorrectly validates certificate expiration boundaries, allowing certificates to be accepted at their exact expiration timestamp. RFC 5280 Section 4.1.2.5 defines X.509 certificate validity periods as `notBefore <= Current Time < notAfter`, meaning a certificate becomes invalid at the `notAfter` timestamp. However, the implementation uses a strict inequality check (`validation_time > not_after`) instead of the RFC-compliant check (`validation_time >= not_after`), creating a one-second window where expired certificates are accepted.

The function validates AWS Nitro Enclave attestation certificate chains during the COSE signature verification process. When `verify_cose_signature` is called, it obtains the current system time and passes it to `verify_cert_chain`, which then calls `verify_cert_validity` for each certificate in the chain (attestation certificate, intermediate certificates, and the AWS root certificate).

```
fn verify_cert_validity(cert: &X509Certificate, validation_time: i64) -> Result<()>
{
    let not_before = cert.validity().not_before.timestamp();
    let not_after = cert.validity().not_after.timestamp();

    if validation_time < not_before {
        return Err(anyhow!("Certificate not yet valid"));
    }
    if validation_time > not_after {
        return Err(anyhow!("Certificate expired"));
    }

    Ok(())
}
```

Figure 18.1: Certificate validity check with incorrect boundary condition
(`tee/enclave/src/attestation/verification.rs`)

The `notBefore` boundary check correctly uses strict inequality (`validation_time < not_before`), rejecting certificates before their validity period begins. However, the

notAfter check should use `validation_time >= not_after` to reject certificates at or after their expiration timestamp, as specified by RFC 5280.

Recommendations

Short term, change the expiration check in `verify_cert_validity` from `validation_time > not_after` to `validation_time >= not_after` to reject certificates at or after their expiration timestamp.

Long term, add test cases that verify boundary conditions for certificate validity periods, including tests that validate rejection at the exact notBefore and notAfter timestamps.

References

- [RFC 5280 Section 4.1.2.5: Validity](#)

19. Template injection vulnerabilities in GitHub Actions workflows

Severity: Low

Difficulty: Medium

Type: Data Validation

Finding ID: TOB-HLTEE-19

Target: `.github/workflows/build-ami.yml`, `.github/workflows/deploy.yml`

Description

GitHub Actions allows workflows to define template expansions, which occur within special `${{ ... }}` delimiters. These expansions happen before workflow and job execution, meaning the expansion of a given expression appears verbatim in whatever context it was performed in. Multiple GitHub Actions workflows directly interpolate user-controlled workflow inputs into shell commands, creating potential template injection vulnerabilities. These vulnerabilities allow an attacker with workflow dispatch permissions to execute arbitrary commands on GitHub Actions runners.

For example, the `deploy.yml` workflow accepts `version`, `environment`, and `ami_id` inputs and interpolates them into multiple shell commands.

```
- name: Get Manifest from S3
  id: manifest
  run: |
    VERSION="${{ inputs.version }}"
    ENVIRONMENT="${{ inputs.environment }}"
```

Figure 19.1: Version validation occurs after interpolation
(`tee/.github/workflows/deploy.yml`)

An attacker can inject shell metacharacters such as backticks, command substitution syntax, or newlines to execute arbitrary commands. The `environment` input is particularly risky because it accepts any string value and is constrained only by a choice list in the UI, which could potentially be bypassed when triggering the workflow via API.

Exploit Scenario

Eve is a developer with write access to the repository and permissions to trigger workflow dispatch. Eve triggers the `build-ami.yml` workflow with a malicious value. The GitHub Actions runner executes the command, exfiltrating the `GITHUB_TOKEN` to Eve's server.

Recommendations

Short term, **fix all the unsanitized template injection** issues present in the codebase.

Long term, add **zizmor** to the CI/CD pipeline.

20. Unpinned external GitHub CI/CD action versions

Severity: Low

Difficulty: High

Type: Auditing and Logging

Finding ID: TOB-HLTEE-20

Target: `.github/workflows/*.yaml`

Description

Several GitHub Actions workflows in the tee repository use third-party actions pinned to a tag or branch name instead of a full commit SHA, as **recommended by GitHub**. This configuration enables repository owners to silently modify the actions. A malicious actor could use this ability to tamper with an application release or leak secrets.

```
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v2

- name: Build Enclave Docker Image
  uses: docker/build-push-action@v4
```

Figure 20.1: `tee/.github/workflows/build.yaml`

The following actions are owned by GitHub organizations or individuals that are not affiliated directly with HyperLink:

- Swatinem/rust-cache@v2
- actions/checkout@v4
- actions/stale@v9
- actions/upload-artifact@v4
- anthropics/claude-code-action@v1
- aws-actions/configure-aws-credentials@v4
- docker/build-push-action@v4
- docker/setup-buildx-action@v2
- dtolnay/rust-toolchain@nightly
- dtolnay/rust-toolchain@stable
- foundry-rs/foundry-toolchain@v1
- hashicorp/setup-packer@main
- hashicorp/setup-terraform@v3
- slackapi/slack-github-action@v2.0.0
- taiki-e/install-action@nextest
- taiki-e/install-action@v2

Exploit Scenario

An attacker gains unauthorized access to the account of a GitHub action owner. The attacker manipulates the action's code to steal GitHub credentials and compromise the repository.

Recommendations

Short term, pin each third-party action to a specific full-length commit SHA, as [recommended by GitHub](#). Additionally, [configure Dependabot](#) to update the actions' commit SHAs after reviewing their available updates.

Long term, add [zizmor](#) to the CI/CD pipeline.

21. Potential privilege escalation due to insecure Dockerfile configurations

Severity: Low

Difficulty: Low

Type: Configuration

Finding ID: TOB-HLTEE-21

Target: `parent/\*`, `prover-enclave/Dockerfile`

Description

Failure to specify a user in a Dockerfile can result in a program within the container running as the root user, posing a significant security risk. If an attacker gains control over a process running with elevated privileges, they may be able to compromise the entire container.

```
CMD ["hyperlink-http-egress"]
```

Figure 21.1: *tee/parent/http-egress/Dockerfile*

This practice was found in multiple locations in the codebase:

- parent/http-egress/Dockerfile
- parent/kms-proxy/Dockerfile
- parent/operator/Dockerfile
- parent/prover-proxy/Dockerfile
- parent/query-cache/Dockerfile
- parent/relay/Dockerfile
- parent/storage/Dockerfile
- parent/ws-tunnel/Dockerfile
- prover-enclave/Dockerfile

Exploit Scenario

An external attacker exploits a vulnerability in a container and enacts privilege escalation since the container is running as root. This includes user data exfiltration, malicious code execution, or container escapes.

Recommendations

Short term, ensure that the final USER directive in all Dockerfiles specifies a non-root user, when appropriate.

Long term, ensure that the Dockerfiles follow Docker's [best practices](#), and consider running static analysis tools such as Semgrep with public sets of rules to consistently uncover such issues.

22. GitHub Actions role can modify KMS key policies, bypassing attestation requirements

Severity: High

Difficulty: Medium

Type: Access Controls

Finding ID: TOB-HLTEE-22

Target: infrastructure/terraform/modules/github-oidc/main.tf

Description

The GitHub Actions CI/CD role has the `kms:PutKeyPolicy` permission, which allows a compromised workflow to modify the KMS key policy and bypass PCR attestation requirements. The KMS key policy is the primary defense mechanism that ensures that only enclaves with specific PCR measurements can decrypt secrets. The CI/CD role's ability to modify this policy undermines the attestation security model by allowing removal of PCR validation conditions, enabling decryption from non-enclave contexts or unauthorized enclave builds.

The GitHub Actions role includes broad KMS management permissions, including the ability to modify key policies:

```
resource "aws_iam_role_policy" "kms" {
  name = "KMSManagement"
  role = aws_iam_role.cicd.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        ...
        {
          Sid      = "KMSManageHyperlinkKeys"
          Effect   = "Allow"
          Action   = [
            "kms:EnableKey",
            "kms:DisableKey",
            "kms:EnableKeyRotation",
            "kms:PutKeyPolicy",
            "kms:ScheduleKeyDeletion",
            "kms:CancelKeyDeletion",
            "kms:TagResource",
            "kms:UntagResource",
            "kms:CreateGrant"
          ]
          Resource = "arn:aws:kms:${var.region}:${local.account_id}:key/*"
        }
      ]
    ]
  })
}
```

```

    Condition = {
      StringEquals = {
        "aws:ResourceTag/Project" = "hyperlink"
      }
    }
  },
},

```

*Figure 22.1: GitHub Actions role has kms:PutKeyPolicy permission
(tee/infrastructure/terraform/modules/github-oidc/main.tf)*

The legitimate KMS key policy includes PCR0 attestation validation to ensure that only enclaves with the correct image measurement can decrypt secrets. However, an attacker with access to GitHub Actions workflows can use the kms:PutKeyPolicy permission to replace this policy with one that removes the PCR attestation condition entirely.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::ACCOUNT_ID:role/EC2_INSTANCE_ROLE"
      },
      "Action": "kms:Decrypt",
      "Resource": "*"
    }
  ]
}

```

Figure 22.2: Malicious policy without attestation requirements

This modified policy allows the parent EC2 instance role to decrypt enclave secrets without providing any attestation document, completely bypassing the TEE security model. The parent VM can then extract Fordefi signing credentials, CipherOwl API keys, and database encryption keys without running code inside an enclave.

Exploit Scenario

Eve submits a pull request adding a seemingly benign GitHub Actions workflow that hides a malicious step modifying the KMS key policy. After the workflow is merged and executed, it uses `aws kms put-key-policy` to remove the PCR attestation condition from the mainnet KMS key, allowing the parent EC2 instance role to decrypt secrets without attestation.

Recommendations

Short term, remove the kms:PutKeyPolicy and secretsmanager:PutResourcePolicy permissions from the GitHub Actions CI/CD role by updating the Terraform configuration at lines 359 and 413. If KMS key policy updates are needed during deployment, implement a

separate manual approval process where an AWS administrator with MFA applies policy changes after verifying PCR values from build artifacts.

Long term, add CloudWatch alarms that trigger on any PutKeyPolicy API calls to detect unauthorized modifications. Create distinct IAM roles for different CI/CD purposes with separation of concerns, and enable AWS CloudTrail to create an immutable audit log of all KMS policy changes. Consider storing all build environment details (including Terraform configuration files) in a public S3 bucket with a legal hold on it in order to allow public monitors to track potential changes to the key policies.

23. HTTP rate limiter configured but never invoked in request handlers

Severity: Low

Difficulty: Low

Type: Configuration

Finding ID: TOB-HLTEE-23

Target: parent/relay/src/main.rs, parent/relay/src/core/http.rs

Description

The HTTP relay service initializes a rate limiter using the governor crate but never invokes it in any request handler. The rate limiter is created during service startup using the `rate_limit_rps` configuration parameter and stored in the `RelayState` shared state. However, neither the `exchange_handler` function (which processes trading requests) nor the `enclave_handler` function (which forwards arbitrary enclave requests) calls the rate limiter's `check` method to enforce rate limits. As a result, the `max_requests_per_ip` configuration parameter has no effect on actual request handling.

The rate limiter is initialized in `main.rs`, as shown below, creating a token bucket rate limiter with a per-second quota based on the configured `rate_limit_rps` value.

```
let rate_limit = std::num::NonZeroU32::new(args.rate_limit_rps)
    .context("rate_limit_rps must be greater than zero")?;
let rate_limiter = Arc::new(RateLimiter::direct(Quota::per_second(rate_limit)));
```

*Figure 23.1: Rate limiter initialization with per-second quota
(tee/parent/relay/src/main.rs)*

The rate limiter is included in the `RelayState` structure that is passed to all HTTP handlers. However, the `exchange_handler` function processes requests without checking the rate limiter. The function immediately increments the metrics counter and proceeds to forward the request to the enclave without any rate limit validation.

Similarly, the `enclave_handler` function does not invoke rate limiting, processing all requests that arrive without enforcing the configured rate limits.

Without functional rate limiting, a single client can send unlimited requests to the relay service, potentially exhausting vsock connection pool capacity or overwhelming the enclave with excessive load. While the vsock connection pool has a maximum connection limit of 32, this provides protection only against connection exhaustion, not against request flooding from a single connection or rapid connection cycling.

Exploit Scenario

Alice begins sending high-frequency requests to the exchange endpoint. Because the rate limiter is never invoked, all of Alice's requests are forwarded to the enclave without throttling. The vsock connection pool attempts to handle the request flood by reusing connections from its pool of 32 maximum connections. However, the enclave's engine becomes overwhelmed with the volume of incoming requests, causing legitimate users' requests to experience increased latency.

Recommendations

Short term, add rate limiting middleware to the HTTP router that checks the rate limiter before processing requests. Extract the client's IP address from the request headers and use it as the rate limiting key. When the rate limiter check fails, return HTTP 429 Too Many Requests with a `Retry-After` header indicating when the client can retry.

Long term, implement comprehensive integration tests that verify rate-limiting behavior by sending requests at rates both below and above the configured limit, confirming that excess requests receive 429 responses.

24. Prover proxy lacks CID validation, message size limits, and connection limits

Severity: **Medium**

Difficulty: **Medium**

Type: Access Controls

Finding ID: TOB-HLTEE-24

Target: parent/prover-proxy/src/main.rs

Description

The prover proxy service, which forwards vsock connections from the main enclave to the prover enclave, lacks three critical security controls that are implemented on other vsock endpoints in the system. First, the proxy accepts connections from any CID without validation. Second, it imposes no limits on message sizes during bidirectional data transfer. Third, it spawns unlimited connection handlers without enforcing a maximum concurrent connection limit.

The proxy binds to VMADDR_CID_ANY (u32::MAX) to accept connections from any vsock CID, as shown in figure 24.1. This contrasts with the client RPC server (port 5000) and operator RPC server (port 5001), which strictly validate that connections originate from the parent VM (CID 3) and reject all other connection attempts.

```
// Listen on VMADDR_CID_ANY (u32::MAX or -1) to accept from any enclave
let listen_addr = VsockAddr::new(u32::MAX, args.listen_port);
let mut listener = VsockListener::bind(listen_addr).context("Failed to bind vsock listener")?;
```

*Figure 24.1: Proxy accepts connections from any CID
(tee/parent/prover-proxy/src/main.rs)*

Additionally, the connection handler uses unbounded bidirectional copying without size limits, as shown in figure 24.2. The `tokio::io::copy_bidirectional` function will transfer unlimited data between the source and target streams. In comparison, the client RPC server enforces a 1 MB message size limit and rejects larger messages before processing them.

```
let (bytes_to_target, bytes_from_target) =
    tokio::io::copy_bidirectional(&mut source, &mut target).await?;
```

*Figure 24.2: Unbounded bidirectional data transfer
(tee/parent/prover-proxy/src/main.rs)*

Finally, the proxy spawns a new tokio task for each incoming connection without any semaphore-based limiting, as shown in figure 24.3. The client RPC server maintains a semaphore with 100 maximum concurrent connections, and the operator RPC server uses a limit of 10 connections. When these limits are reached, new connections are rejected until capacity becomes available.

```
tokio::spawn(async move {  
    if let Err(e) = handle_connection(stream, peer, target_cid, target_port).await {  
        error!(peer_cid = peer.cid(), error = %e, "Connection failed");  
    }  
});
```

Figure 24.3: Unlimited connection spawning (*tee/parent/prover-proxy/src/main.rs*)

These missing security controls create attack vectors for resource exhaustion and lateral movement within the vsock network.

Exploit Scenario

Eve compromises the parent VM and gains the ability to create arbitrary vsock connections. She connects to the prover proxy (port 5007) from an attacker-controlled CID within the parent VM. Because the proxy accepts connections from any CID, her connection is accepted. Eve then floods the proxy with connections, spawning thousands of connection handler tasks. Each task establishes a corresponding connection to the prover enclave (CID 17, port 5008). The prover enclave's connection handler also lacks a connection limit, so it accepts all incoming connections. Eve exhausts system resources on both the parent VM and the prover enclave through this connection flooding attack.

Recommendations

Short term, add CID validation to the prover proxy. Implement message size limits by replacing the unbounded `copy_bidirectional` call with a size-checked transfer function that enforces maximum sizes for requests and responses, matching the prover enclave's existing limits. Add a semaphore-based connection limit for concurrent connections to prevent resource exhaustion.

Long term, add integration tests that verify each vsock endpoint correctly rejects unauthorized CIDs, oversized messages, and excessive connections.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Issues

This appendix contains findings that do not have immediate or obvious security implications. However, addressing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Data length check is too permissive.** The ciphertext contains a 16-byte authentication tag which is unaccounted for (since the `HEADER_LEN` is defined to be 13 bytes); the check below should ensure that the data is at least `HEADER_LEN + 16`.

```
ensure!(data.len() >= HEADER_LEN, "data too short");
```

Figure B.1: `tee/enclave/src/crypto/encryption.rs`

The test `data_too_short_fails` indicates a slightly incorrect understanding of the above size check; an incorrect authentication tag should fail invariably since the tag is missing and the result should not be dependent on the size of the tag.

```
// This may pass or fail depending on GCM tag requirements
// but should not panic
let _ = enc.decrypt(&short);
```

Figure B.2: `tee/enclave/src/crypto/encryption.rs`

Similarly, the function `decrypt` excerpted below is also too permissive when checking the length of the data.

```
/// Decrypts data using AES-256-GCM with the shared secret.
/// Input format: nonce (12 bytes) || ciphertext
pub fn decrypt(shared_secret: &[u8; 32], data: &[u8]) -> Result<Vec<u8>> {
    if data.len() < 12 {
        return Err( anyhow!("Data too short for nonce") );
    }
}
```

Figure B.3: `tee/common/src/crypto.rs`

- **secret should be an `EphemeralSecret` instead of `StaticSecret`.** The key exchange implementation uses a `StaticSecret` for generating ephemeral key pairs, but should use the `x25519-dalek` crate's `EphemeralSecret` type instead, since this key pair is ephemeral. `EphemeralSecret` provides automatic zeroization on drop and is specifically designed for short-lived keys in Diffie-Hellman exchanges, while `StaticSecret` is intended for long-term keys. Using the wrong type misses important security properties and does not clearly communicate the intended lifetime of the key material.

```
/// Generates a new X25519 keypair.
pub fn generate_keypair() -> (StaticSecret, PublicKey) {
```

```
let secret = StaticSecret::random_from_rng(rand::thread_rng());
let public = PublicKey::from(&secret);
(secret, public)
}
```

Figure B.4: `tee/common/src/crypto.rs`

- **Inconsistent comparison operators between related code blocks.** Two related code blocks that perform similar expiration validation logic use different comparison operators: one uses the strict inequality `>`, while the other uses the inclusive inequality `>=`. This inconsistency suggests that the boundary condition handling may be incorrect in one of the locations, potentially causing edge-case bugs where values exactly at the boundary are handled differently than intended. The logic should be reviewed to determine which operator is correct and applied consistently. The two code blocks are:

```
if now < details.validFrom || now >= details.validUntil {
```

Figure B.5: `tee/enclave/src/chain/contract_integration.rs`

and

```
if now < status.valid_from || now > status.valid_until {
```

Figure B.6: `tee/enclave/src/chain/contract_integration.rs`

- **Fail open behavior may be dangerous.** The error handling logic in the CipherOwl screening integration implemented in the function `is_high_risk` appears to fail open when errors occur, meaning that deposits may proceed without proper compliance screening if the CipherOwl service is unavailable or returns an error. This creates a security risk where malicious actors could exploit service disruptions or intentionally trigger errors to bypass sanctions screening. The system should either fail closed (reject deposits on screening errors) or implement a robust fallback mechanism with appropriate logging and alerting.

```
/// Returns true if the address is flagged as high risk, false otherwise.
/// Errors are logged and treated as "not high risk" (fail-open).
pub async fn is_high_risk(&self, address: &str) -> Result<bool> {
    let response = self.screen_address(address).await?;
    if response.found_risk {
        warn!(
            "High risk wallet: {} on {} - {:?}",
            address, self.chain_name, response.explanations
        );
    }
    Ok(response.found_risk)
}
```

Figure B.7: `tee/enclave/src/io/transfer_monitor/cipherowl.rs`

- **Attestation is parsed twice inefficiently.** The prover enclave attestation verification flow parses the attestation document twice (in the file `enclave/src/clients/prover.rs`): first to extract and validate the nonce field, and then again to parse the complete document structure. This double-parsing is inefficient and increases code complexity. The logic should be refactored to parse the attestation document once, extract all required fields including the nonce during the single parse operation, and then perform validation on the parsed structure.
- **Two vector size hints are incorrect.** Two vector size hints are set to incorrect values. In the encoding of the domain separator in the function `compute_domain_separator`, the correct size should be 160 bytes:

```
let mut encoded = Vec::with_capacity(128);
```

Figure B.8: `tee/enclave/src/crypto/mod.rs`

and in the encoding of a `struct_hash`, it should be 256.

```
let mut encoded = Vec::with_capacity(224);
```

Figure B.9: `tee/enclave/src/crypto/mod.rs`

C. Automated Analysis Tool Configuration

We used the following tools to perform automated testing of the codebase.

C.1. Clippy

The Rust linter **Clippy** can be installed using `rustup` by running the command `rustup component add clippy`. Invoking `cargo clippy -- -W clippy::pedantic` in the root directory of the project runs the tool with the pedantic ruleset.

Clippy results from the regular set of rules should always be fixed.

```
# run clippy in regular and pedantic mode and output to SARIF
cargo clippy --message-format=json | clippy-sarif > clippy.sarif
cargo clippy --message-format=json -- -W clippy::pedantic -Wclippy::or_fun_call |
clippy-sarif > clippy_pedantic.sarif
```

Figure C.1: The invocation commands used to run Clippy in the codebase

Converting the output to the SARIF file format (e.g., with **clippy-sarif**) allows easy inspection of the results within an IDE (e.g., using VSCode's **SARIF Explorer** extension).

C.2. Dylint

Dylint is a tool for running Rust lints from dynamic libraries, similar to Clippy..

```
cargo dylint --no-deps --git https://github.com/trailofbits/dylint --pattern
examples/general -- --message-format=json | clippy-sarif > general.sarif

cargo dylint --no-deps --git https://github.com/trailofbits/dylint --pattern
examples/supplementary -- --message-format=json | clippy-sarif > supplementary.sarif
```

Figure C.2: The command used to run Dylint on the project

C.3. Semgrep

We used the static analyzer **Semgrep** to search for weaknesses in the source code repository.

```
semgrep --metrics=off --sarif --config "p/trailofbits"
```

*Figure C.3: The invocation command used to run Semgrep with Trail of Bits' **public rules***

The **Testing Handbook** provides detailed information about how to set up Semgrep, as well as how to add it to continuous integration.

C.4. zizmor

Zizmor is a static analysis tool for GitHub Actions. It can be installed using `cargo install --locked zizmor`. We ran zizmor with the following command:

```
zizmor .github/workflows/* --format sarif > zizmor_pedantic.sarif
```

Figure C.4: Command used to run zizmor and output to SARIF

C.5. cargo-edit

cargo-edit allows developers to quickly find outdated Rust crates. The tool can be installed with the `cargo install cargo-edit` command, and the `cargo upgrade --incompatible --dry-run` command can be used to find outdated crates.

C.6. cargo-audit

The **cargo-audit** Cargo plugin identifies known vulnerable dependencies in Rust projects. It can be installed using `cargo install cargo-audit`. To run the tool, run `cargo audit` in the crate root directory.

D. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

From January 29 to January 30, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the HyperLink team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

The remediation code for these findings produced substantial code changes and added considerable complexity to the codebase. Several PRs included modifications beyond the scope of the original security findings, making a comprehensive review of all changes infeasible within the fix review timeframe. This review therefore focused on verifying that the core vulnerability described in each finding had been addressed, while acknowledging that ancillary changes may have introduced new vulnerabilities and may warrant additional scrutiny in the future.

In summary, of the 24 issues described in this report, HyperLink has resolved 22 issues and has partially resolved two issues. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	Prover enclave public key not validated against attested public key	Medium	Resolved
2	Unauthenticated ephemeral key exchange enables parent MITM attacks	Medium	Partially Resolved
3	Attestation verification missing from important operator flows	Medium	Resolved
4	Deficiencies in the enclave registration and attestation processes	High	Partially Resolved
5	KMS key policy only validates PCRO	Low	Resolved
6	X25519 shared secret used directly without key derivation function	Low	Resolved

7	Missing length validation in key exchange response may cause denial of service	Low	Resolved
8	Incomplete attestation document validation against AWS Nitro specification	Medium	Resolved
9	Missing extension constraint validation in certificate chain verification	Informational	Resolved
10	Storage download accepts unbounded allocation size	Medium	Resolved
11	Storage rollback attack on database snapshots	High	Resolved
12	Backup timestamp not validated during recovery	Informational	Resolved
13	Potential integer truncation in database sync	Informational	Resolved
14	Missing secret version validation may allow stale credential usage	Low	Resolved
15	Clock source verification only logs warning	Low	Resolved
16	Documented client-to-enclave encryption not implemented	Medium	Resolved
17	Missing signature algorithm validation in X.509 chain verification	Informational	Resolved
18	Certificate validity boundary check non-compliance with RFC 5280	Informational	Resolved
19	Template injection vulnerabilities in GitHub Actions workflows	Low	Resolved
20	Unpinned external GitHub CI/CD action versions	Low	Resolved
21	Potential privilege escalation due to insecure Dockerfile configurations	Low	Resolved

22	GitHub Actions role can modify KMS key policies, bypassing attestation requirements	High	Resolved
23	HTTP rate limiter configured but never invoked in request handlers	Low	Resolved
24	Prover proxy lacks CID validation, message size limits, and connection limits	Medium	Resolved

Detailed Fix Review Results

TOB-HLTEE-1: Prover enclave public key not validated against attested public key

Resolved in [PR #614](#). The vulnerable X25519 + AES-GCM relying on per-request ephemeral keys was replaced with the Noise NK protocol, which uses the attested public key during handshake.

TOB-HLTEE-2: Unauthenticated ephemeral key exchange enables parent MITM attacks

Partially resolved in [PR #614](#). The per-request ephemeral key exchange was replaced with session reuse and the Noise NK protocol. However, the implementation has limitations: (1) Mutual authentication is architecturally broken due to a circular dependency; the prover cannot validate the main enclave PCRs at build time because the main enclave PCRs depend on the prover's code, which depends on the main's PCRs. (2) Channel binding is incorrectly implemented; the attestation hash is used as Noise prologue (binding the handshake to the attestation). Proper channel binding requires signing the handshake hash to cryptographically bind authentication to the specific Noise session. (3) Unnecessary protocol complexity; the session establishment protocol consists of four round-trips and introduces an additional static ECDSA key pair held by the main enclave, all of which significantly increase the attack surface and implementation complexity. While the Noise NK protocol appears to prevent the original per-request key substitution attack, the fix introduces new architectural weaknesses and potential protocol vulnerabilities that may warrant a deeper review.

TOB-HLTEE-3: Attestation verification missing from important operator flows

Resolved in [PR #614](#). Attestation verification is now implicitly enforced through session establishment rather than requiring explicit per-operation calls. All requests flow through `send_encrypted`, which ensures a session exists, and every session establishment performs attestation verification, Noise handshake with the attested key, and mutual authentication.

TOB-HLTEE-4: Deficiencies in the enclave registration and attestation processes

Partially resolved in [PR #618](#). The operator-supplied PCR parameters were removed, and the function `generate_self_attestation` was implemented to derive PCR values directly from attestations of a running enclave. However, a number of important pieces are still missing, including on-chain attestation verification, binary transparency, and public verification APIs.

TOB-HLTEE-5: KMS key policy only validates PCR0

Resolved in [PR #619](#). Documentation and tooling were updated to require validation of all three PCR values (PCR0, PCR1, PCR2) in KMS key policies. The KMS policy template in `docs/AWS_NITRO_SETUP.md` was updated to include attestation conditions for all three PCRs with security warnings explaining the vulnerability, and a new automation script

scripts/generate_kms_policy.sh was added to extract PCR values from build artifacts and generate complete KMS policies.

TOB-HLTEE-6: X25519 shared secret used directly without key derivation function

Resolved in [PR #617](#). The custom crypto module that used X25519 shared secrets directly as AES-256-GCM encryption keys was completely removed along with its dependencies. The functionality was replaced by migrating to the Noise Protocol Framework, which provides proper key derivation.

TOB-HLTEE-7: Missing length validation in key exchange response may cause denial of service

Resolved in [PR #616](#). Length validation was added to the handshake response path by introducing a MAX_HANDSHAKE_RESPONSE_SIZE constant (1KB) and refactoring response reading to use a read_response_with_limit helper function that validates length before allocation.

TOB-HLTEE-8: Incomplete attestation document validation against AWS Nitro specification

Resolved in [PR #620](#). Complete attestation document validation was implemented per AWS Nitro NSM API specification, addressing all missing validations outlined in the finding.

TOB-HLTEE-9: Missing extension constraint validation in certificate chain verification

Resolved in [PR #621](#). Three missing X.509 extension constraints were added to certificate chain verification per RFC 5280 and AWS Nitro specifications: CA certificates now validate pathLenConstraint against actual chain depth, leaf certificates are rejected if pathLenConstraint is defined, and leaf certificates are rejected if keyCertSign is present in Key Usage.

TOB-HLTEE-10: Storage download accepts unbounded allocation size

Resolved in [PR #632](#). Added MAX_REDB_DOWNLOAD_SIZE constant (256 MB) and validate_download_size helper to reject oversized or empty downloads from the untrusted parent storage service.

TOB-HLTEE-11: Storage rollback attack on database snapshots

Resolved in [PR #633](#). Three defense layers were introduced to address potential storage rollback attacks: snapshot metadata (including timestamp, epoch, balance_root, position_root) is bound as Additional Authenticated Data (AAD) in the AES-256-GCM encryption of snapshots so decryption fails if metadata is tampered; on-chain validation queries the StateManager contract during recovery to verify the snapshot's epoch and Merkle roots match committed on-chain values (on mainnet only); and timestamp validation rejects zero or future timestamps with consistency checks between wire protocol responses.

TOB-HLTEE-12: Backup timestamp not validated during recovery

Resolved in [PR #633](#). The timestamp field transmitted in backup metadata is now validated during recovery through `validate_snapshot_metadata`.

TOB-HLTEE-13: Potential integer truncation in database sync

Resolved in [PR #638](#). All database sync protocol length fields were upgraded from u32 (4 bytes) to u64 (8 bytes) in both the enclave sender and storage service receiver.

TOB-HLTEE-14: Missing secret version validation may allow stale credential usage

Resolved in [PR #640](#). A build-time `MINIMUM_SECRET_VERSION` constant was added and the function `EnclaveSecrets::validate_version` was implemented to check loaded secrets against this minimum. Note that no constraints on the maximum version are enforced.

TOB-HLTEE-15: Clock source verification only logs warning

Resolved in [PR #634](#). The `verify_clock_source` function was changed to return a `Result` type and now fails enclave initialization when the clock source cannot be verified or is not `kvm-clock/kvm-clocksource` in enclave mode.

TOB-HLTEE-16: Documented client-to-enclave encryption not implemented

Resolved in [PR #637](#). E2E encryption was implemented using X25519 + AES-256-GCM. Signed handshake responses were added to prevent relay MITM attacks using the enclave's long-term signing key. During the audit, we noted that the hash to be signed is computed without domain separation, which, theoretically, could be used as a transaction signing oracle. Although the construction seems secure since all inputs are enclave-controlled, we still recommend adding a domain separation tag. Also, plaintext requests remain supported for backwards compatibility. Thus, the security depends on client-side implementation: clients must verify handshake signatures against the on-chain `EnclaveKeyRegistry` and use E2E encryption to achieve confidentiality.

TOB-HLTEE-17: Missing signature algorithm validation in X.509 chain verification

Resolved in [PR #635](#). Signature algorithm validation was added via a new `verify_cert_signature_algorithm` function that validates that the fields match the expected OID.

TOB-HLTEE-18: Certificate validity boundary check non-compliance with RFC 5280

Resolved in [PR #636](#). The certificate expiration check was changed to `validation_time >= not_after`, correctly implementing RFC 5280 Section 4.1.2.5's exclusive upper bound requirement.

TOB-HLTEE-19: Template injection vulnerabilities in GitHub Actions workflows

Resolved in [PR #641](#). All potentially dangerous template expressions were moved out of `run` blocks into step-level `env` blocks across the relevant YAML files.

TOB-HLTEE-20: Unpinned external GitHub CI/CD action versions

Resolved in [PR #641](#). All 17 third-party GitHub Actions across eight workflow files were pinned to full commit SHAs instead of mutable version tags.

TOB-HLTEE-21: Potential privilege escalation due to insecure Dockerfile configurations

Resolved in [PR #642](#). A non-root appuser (system user/group) was added to all nine Dockerfiles via the USER directive, preventing containers from running as root and mitigating privilege escalation risk if a containerized service is compromised.

TOB-HLTEE-22: GitHub Actions role can modify KMS key policies, bypassing attestation requirements

Resolved in [PR #643](#). The kms:PutKeyPolicy, secretsmanager:PutResourcePolicy, and secretsmanager:DeleteResourcePolicy permissions were removed from the GitHub Actions OIDC role's IAM policies, preventing a compromised CI/CD workflow from rewriting KMS key policies or secret access policies to bypass PCR attestation requirements.

TOB-HLTEE-23: HTTP rate limiter configured but never invoked in request handlers

Resolved in [PR #639](#). IP-based rate limiting middleware was implemented and applied to all HTTP routes, returning HTTP 429 with Retry-After headers when limits are exceeded, and a background cleanup task was added to prevent memory growth from rate limiter state.

The client also provided the following context for this finding's fix:

We did not find this exploit scenario valid as we've moved rate limiting to CloudFront native protections, but we've implemented the suggested fixes for completeness.

TOB-HLTEE-24: Prover proxy lacks CID validation, message size limits, and connection limits

Resolved in [PR #631](#). CID validation was added to restrict connections to the main enclave (CID 16), message size limits were implemented using length-prefixed forwarding (100 MiB request / 50 MiB response), and a semaphore-based connection limit of eight concurrent connections was enforced to prevent resource exhaustion.

E. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	Description
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to HyperLink under the terms of the project statement of work and has been made public at HyperLink's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.